SPACE

Media Server

Complete Documentation

Architecture - API Reference - Security - Extensions - Operations

Generated from source

Table of Contents

1. Project Overview
2. Security Posture
3. Security Audit
4. Getting Started
5. Architecture
6. Configuration
7. Database
8. Deployment
9. API Reference
10. WebSocket Protocol
11. Media Graph
12. Security Model
13. Mood Search & Smart Collections
14. Heuristic Engine
15. Card Economy
16. Automation & Webhooks
17. Plugin Guide
18. Python Plugins
19. Streaming Plugins
20. Stremio Extensions
21. Web Sources
22. Adding a Source
23. Telemetry & Monitoring
24. Torrents
25. RiveStream
26. Unified AI Security Plan

SPACE -- Production-Grade Media Platform

A modular monolith media server built with **Rust / Axum / SQLx / PostgreSQL**:
catalog (movies · series · anime), full-text & vibe search, streaming with HLS,
watch tracking, social, watch parties, chat over WebSockets -- plus a **runtime
plugin system** for metadata sources (TMDB, IMDb via OMDb, AniList, or your own).

```
+------------- clients --------------+
|  web SPA (./web)   mobile/TV/SDKs  |
+--------------?---------------------+
        nginx TLS proxy (ops/)
               |
+--------------?-----------------------------------------+
|                     Axum HTTP + WS                      |
| auth | catalog | search | tracking | streaming | playback |
| chat/hub | social | parties | intelligence | plugins     |
| ratelimit | metrics | telemetry | OpenAPI               |
+-------?---------------?------------------?---------------+
   PostgreSQL       Redis (cache/RL)    ffmpeg/ffprobe jobs
```

Quick start

```
docker compose up -d postgres redis      # infrastructure
cp .env.example .env                      # set DATABASE_URL etc.
cargo run                                 # migrations run automatically
# -> http://localhost:8080          (web client)
# -> http://localhost:8080/docs     (Swagger UI)
# -> http://localhost:8080/admin.html (plugin console)
```

Admin bootstrap: set `ADMIN_EMAIL` + `ADMIN_PASSWORD`, they are provisioned on
first boot. Library scanning: point `LIBRARY_ROOTS` at your media, then
`POST /api/admin/library/scan`.

Feature map

Area	Highlights
Auth	Argon2id hashes, opaque rotating refresh sessions (hashed at
API keys	`sk_` scoped machine tokens (`read`/`write`/`downloads`) via
Devices	Registered device list w/ platform + last-seen; download lin
Catalog	Unified `media` (**movies · series · books**), seasons/episo
Metadata	Built-ins: **TMDB**, **IMDb (OMDb)**, **AniList**; runtime H
Search	FTS + trigram fuzzy (auto ILIKE fallback on CockroachDB), di
Query parser	`action 2020-2025 1080p english` -> structured filters (`/ap
Tracking	Progress beacons w/ auto-complete, continue-watching, status
Streaming	Storage abstraction, scanner (video **+ epub/pdf/mobi/azw3/f
Playback	Sessions + heartbeats, HLS ladder transcodes, thumbnails, re
Social	Reviews, threaded comments, reactions, follows (+notificatio

Chat	WebSocket gateway (rooms, typing, presence, receipts), DMs/g
Parties	Join codes, host controls, wall-clock-correct sync state, ev
Notifications	In-app feed + unread count; WS push; triggers on follow & pa
Downloads	HMAC-signed, expiring (<=24 h), device-bound, nonce-budgeted
Lyrics	Multi-provider (Better Lyrics -> LRCLIB -> NetEase -> Genius
YouTube	Five-layer source chain (InnerTube -> Piped -> Invidious ->

Extension system -- Stremio addons, 1:1

SPACE natively speaks the Stremio Addon SDK: install any community addon by manifest url (console -> **Extensions**, with one-click browse of the central index) and its catalogs join unified search while its `/stream` endpoints join playback resolution -- torrents become magnets automatically.

docs/extensions.md

```
curl -X POST http://localhost:8080/api/admin/extensions \
  -H "Authorization: Bearer $TOKEN" -d '{"url":"https://torrentio.strem.fun/manifest.json"}'
```

Differentiators

- **Taste DNA** -- genre affinity, pace & era preferences per user (`GET /api/me/taste`)
- **Heuristic Engine** -- built-in learning from every play, rewatch, rating & favorite: Wrapped recaps, chronotype habits, binge sessions, personas, explainable next-up picks (`GET /api/me/heuristic`) -- see docs/heuristic-engine.md
- **One-call Home** -- `/api/home` composes all rails incl. because-you-watched
- **Vibe search** -- deterministic natural-language mood engine, no external AI
- **Query parser** -- free-form text becomes precise filters
- **Smart collections** -- saved rule queries as living shelves
- **Plugin system** -- metadata + full-library source plugins at runtime

Books

Books are catalog items (`media_type=book`). The scanner ingests `epub pdf mobi azw3 fb2 djvu cbz cbr`; clients track position via `PUT /api/tracking/reading {media_id, position_percent, page}` and resume with `GET /api/me/media/{id}/reading`. Files stream through the same signed-download pipeline.

Source plugins

Beyond metadata search, a plugin with `"kind": "source"` exposes `GET {base}/items` returning `[{id,title,type,year,overview,poster}]`. `POST /api/admin/plugins/{id}/sync` imports the whole library into the catalog with dedupe via external mappings.

Plugins

Register any HTTP service speaking the JSON protocol -- at runtime, from the admin console or the API. Ready-made examples live in `plugins/python/`:

Plugin	Kind	What it does
`space_plugin.py`	source	minimal protocol reference
`leetx_plugin.py`	source	1337x torrents -> magnet streams ([docs](docs/torrents.md))

`rivestream_plugin.py`	source	RiveStream movies/TV players via TMDB ids ([docs](docs/rives
`zstream / cineby / toustream / cubeflix / cinebytv`	source	TMDB-keyed movie & TV frontends ([docs](docs/streaming-plugi
`miruro / animepahe / kaa / animex / anikoto / reanime / lun	source	anime & manga (miruro real HLS; nyaa RSS magnets; luna trend
`myanimelist / simkl / rottentomatoes`	source	catalog/ratings/tracking search bridges
`kissasian / dramanice / pahe / kisskh / goplay / asiaflix /	source	dramas, encodes & passthrough catalogs

```
GET  {base}/search?q=   -> {"results":[{"id","title","overview","year","poster","type"}]}
GET  {base}/details?id= -> {"title","overview","release_date","poster","media_type",
                            "genres":[],"studios":[],"cast":[{"name","character"}]}
```

```
curl -X POST /api/admin/plugins -H "$AUTH" -d '{
  "name":"tmdb-fr", "kind":"metadata",
  "base_url":"https://my-plugin.example.com", "enabled":true }'
```

Plugins appear instantly under `/api/metadata/sources`, work with `/api/metadata/search` and `/api/metadata/import`. Write them in any language.

Observability & operations

- Structured logs (`LOG_FORMAT=json`), request latency histograms, WS gauges, worker metrics -> `/metrics` (Prometheus).
- `/health` (liveness), `/api/ready` (DB deep check), `/api/stats` dashboard.
- Production: `docker-compose.prod.yml` + `ops/nginx.conf` (TLS-ready, WS-aware).
- Backups: `ops/backup.sh` nightly dump w/ retention; `ops/restore.sh` includes restore *verification* steps -- rehearse monthly.
- Load tests: `tests/load/browse_load.js` (500 VU browse mix), `ws_stress.js` (socket storms), `stream_stress.js` (range-req

uest
IO).

Run against staging with thresholds; investigate regressions via `/metrics`.

Documentation

Full docs live in docs/:

- API Reference -- every endpoint with curl examples
- WebSocket Protocol
- Plugin Guide
- Configuration -- Architecture -- Database -- Deployment

Interactive OpenAPI: /docs once the server is running.

Built-in source plugins (native, zero-hop)

Source	Category	What you get
ia-films	movies	Internet Archive public-domain films (streamable MP4s, torre
audius	music	Trending + searchable tracks with direct audio streaming
mangadex	manga	Licensed manga: search, chapter list, full page reader
gutenberg	books	75k public-domain ebooks (read online / EPUB / Kindle)

Sources are server-managed: clients call the unified endpoints and the backend auto-selects providers, ranks results, picks the best stream, and fails over automatically. Raw per-source routes exist only as admin debug tools.

Unified API:

```
GET  /api/search/all?q=&limit=&category=   aggregated results w/ opaque tokens
POST /api/playback/resolve                 { token } -> best stream URL
POST /api/playback/manga/open              { token } -> chapters + first pages
POST /api/admin/sources/{name}/sync        import N items into your catalog
```

Learned web sources -- any site, one field (homelab)

Paste a site url at `/console` -> **Web Sources** (or `POST /api/admin/websources {"site": "https://..."}`) and the bundled galaxy scraper learns the site's search endpoint, result cards and item pages automatically. Learned sites join unified search, playback resolution with failover, health monitoring and sync like any other source.

```
POST /api/admin/websources        { "site": "https://movies.example" }   -> learned
POST /api/admin/websources/{id}/test                                     -> live check
```

See docs/web-sources.md.

Security & Reliability Posture

Maps the audit checklist (Phases 12-15) to concrete mechanisms in this codebase.

Authentication

- Argon2id (default params, per-password salt) -- `src/auth/password.rs`
- Opaque bearer tokens; **only SHA-256 hashes stored**; rotation on refresh with replay detection (`secure_login`, `refresh` in `src/auth/service.rs`)
- HttpOnly + SameSite=Lax (+Secure in prod) session cookies
- Brute-force lockout: per (identifier, IP) sliding window -> timed lock (`login_lockouts`), thresholds via env
- Password policy gate before Argon2 (length + class mixing)
- Password reset: single-use hashed tokens, 30-minute TTL, all sessions revoked on completion; request endpoint is enumeration-safe

Authorization

- Role hierarchy (`user < moderator < admin`) + DB grants for granular permissions (`user_permissions`, `permissions::require`)
- Admin-only surfaces: catalog writes, library scan/probe/transcode/markers, plugin management, bulk user listing
- Object ownership enforced server-side (lists, reviews, comments, sessions)
- WS room joins re-validate membership per event (`chat/ws::authorize_room`)

Input validation & injection

- All SQL through parameterized sqlx bindings (no string interpolation of user data); dynamic filters built with `QueryBuilder` + bound values only
- Length/charset checks on usernames, emails, bodies, emoji, list names
- Path traversal: storage `resolve()` and every file join verify the resolved path

stays under its root; HLS asset route re-checks prefix

Abuse resistance

- Fixed-window rate limiting (Redis INCR w/ memory fallback): global per-IP + strict auth bucket; edge layer adds nginx `limit_req` for defense in depth
- Party capacity limits, private/friends visibility gates, host-only controls

Downloads (Phase 21)

Signed download URLs implement the **industry-standard authorization model**:

- HMAC-SHA256 over `(file, user, device, expiry, nonce)` with a server-only secret (auto-generated, DB-stored); signature verified in constant time

- Hard expiry (default 30 min, cap 24 h) + per-grant hit budget -> link sharing dies quickly; device binding ties a URL to a registered device
- Issuance requires the explicit `download` permission grant or admin role; every issuance is audited (`download_grants`)
- `Content-Disposition: attachment`, `Cache-Control: private, no-store`

Honest limitation: no software can make media *unstealable* -- any client that renders bytes can copy them. Claims of "uncrackable for 1000 years" are marketing, not engineering. What raises real cost for leakage: server-side envelope encryption of offline packages with per-device key wrapping, expiring license blobs with remote kill-switch, and forensic watermarking. These are the M5 roadmap items and follow Widevine-style trust models.

- Config exclusively from environment (`.env` git-ignored); tokens never logged
- `cargo audit` / Dependabot in CI recommended; pin workspace versions

Reliability drills

- Redis down ? cache & rate limiter degrade to in-process fallbacks (tested by stopping redis during load runs)
- ffmpeg missing ? transcode jobs fail *gracefully* with explicit status, strea

ming endpoints stay healthy

- Restart/recovery ? graceful shutdown drains connections; migrations are idempotent; sessions survive restarts (Postgres-backed)

Secrets & dependencies

- Config from environment only; tokens never logged

SPACE -- Full-Project Security Assessment

Scope: entire repository as of this pass. Method: manual static review of every auth path, middleware, money path, crypto construction and query surface, plus executable unit tests for security-critical pure logic.
Result: **all findings below fixed in-tree; 43/43 tests green.**

---

1. Findings & resolutions

ID	Sev	Finding	Status
F-01	High	**Honeypot unreachable for remote attackers.** The gate's lo	**Fixed** -- gate intercepts `/api/admin/debug/dump-secrets`
F-02	High	**Card gate gaps.** `hls_master`, `hls_asset` and `request_d	**Fixed** -- gates added; playback surface now fully covered
F-03	Med	**Multi-statement SQL** in `block_user` (`INSERT ...; DELETE	**Fixed** -- split into two prepared statements
F-04	Med	**Journey filter bypass** -- reverse-direction adaptation st	**Fixed** -- direction-agnostic `!seen` filter
F-05	Low	**Cookie name hard-coded** in console-page gate while config	**Fixed** -- uses `SESSION_COOKIE` constant
F-06	Low	**Unbounded search cache** growth on adversarial query varie	**Fixed** -- 800-entry cap with TTL sweep
F-07	Low	**Invisible presence broadcast** leaked	**Fixed** -- stored, never broadcast
F-08	Info	Advertised endpoints (`storage/analytics`, `files/{id}/event	**Fixed**

2. Threat model -> control matrix

Threat	Control(s)
Credential stuffing / brute force	per-(identity,IP) lockouts · admin escalation bans (4× windo
Stolen bearer/session tokens	HttpOnly cookies · hashed refresh storage · logout-all · hon
Stolen playback grants replayed elsewhere	HMAC grants bound to user+media+quality+expiry **and** TLS/d
Bulk scraping after initial access	segment-scoped child grants (`seg:N`, <=120 s) · one-time 20
Key theft	plaintext keys never stored; HKDF purpose KEKs; AES-256-GCM
Leak tracing	forensic `wm=` watermark binding user+session+hour on every
Direct-to-origin media theft	`X-Origin-Token` gate on key/download endpoints (CDN-only pa
Admin page attacks	loopback default · watchdog lockout · permanent honeypot ban
Site abuse during incident	site-wide lockdown switch (public 503, admin+health alive)
Insider/merchant fraud	purchase approval queue · instant card hold · merchant ban ·
Trial farming	partial unique index -- exactly one trial card per account,
SQL injection	parameterized queries everywhere; dynamic SET clauses built
XSS/injection via client markup	API returns JSON only; no server-rendered user content
DoS on hot paths	bounded caches (search <=800 entries) · circuit breakers · f

3. Crypto inventory

Purpose	Construction	Notes
Access/refresh tokens	random 32 B, SHA-256 hashed at rest	rotation on refresh
Playback/download/card signatures	HMAC-SHA256, constant-time compare	secrets per-purpose in `signing_secrets`
Media keys at rest	AES-256-GCM under HKDF-SHA256 KEKs	fresh 96-bit nonce per wrap
HLS segments	AES-128 content key derived via HKDF from media key	released only against live grants
TOTP	RFC-6238 HMAC-SHA1/30 s/6 digits ±1 step	RFC test vectors unit-tested
Card numbers	body + HMAC checksum group	tamper-evident offline

4. Executable security tests (this pass)

- `watchdog::tests::per_ip_pattern_trips_and_resets` -- proves the detector trips on repeated failures and that reset restores service.
- `cards::tests::checksum_detects_tampering` -- checksum equality, tamper rejection, secret-sensitivity.
- `grants::tests::segment_scope_semantics` -- scoped tickets cover exactly one segment; full-access covers all.
- `keys::tests::aead_roundtrip_and_tamper_rejection` -- GCM auth failure on ciphertext bit-flip and wrong KEK.
- `totp::tests::rfc6238_test_vector` + window/reject cases.
- `watermark::tests::*` -- deterministic codes per hour bucket, URL re-stamping replaces rather than stacks, fragments preserved.
- `aggregator::tests::*` -- preference ranking order, duplicate-title merge keys.

5. Residual risks (documented, by design)

1. **Offline copies**: once a key is delivered, server cannot claw back bytes -- mitigated by 24 h license windows + media-key rotation killing future re-delivery.
2. **Watermark is URL-level**: determined redistributors can strip query strings; pixel-level marking stays a transcode-pipeline hook.
3. **JA4 requires edge cooperation**: without a proxy computing it, fingerprinting falls back to client-supplied values (still binds grants; trust is weaker and documented).
4. **KMS/HSM**: master secret lives in env/generated store; swapping to a KMS only replaces `master_secret()`.
5. **Watchdog memory reset**: a process restart clears an active trip (DB bans persist) -- acceptable; localhost recovery path unaffected.

Getting Started

0. Prerequisites

- Rust 1.85+ (`rustup`)
- PostgreSQL 14+ **or** CockroachDB -- Docker is optional (see "No Docker" below)
- Redis optional -- without it, cache/rate-limiting fall back to in-memory
- `ffmpeg` + `ffprobe` on PATH -- optional, needed for transcoding, thumbnails, subtitle extraction and intro detection

No Docker (plain localhost)

1. Install Postgres natively:
- Windows: `winget install PostgreSQL.PostgreSQL.16`
- macOS: `brew install postgresql@16 && brew services start postgresql@16`
- Linux: `sudo apt install postgresql`
2. Create the database:

```
createdb space

# or: psql -U postgres -c 'CREATE DATABASE space;'
```

3. `.env` needs only:

```
DATABASE_URL=pos
```

```
tgres://postgres:yourpassword@localhost:5432/space
```

4. `cargo run --release` -> open http://localhost:8080

Redis can be added later (`REDIS_URL=redis://localhost:6379`) with zero code changes; everything falls back automatically until then.

0b. Infrastructure (Docker route)

```
docker compose up -d postgres redis
```

CockroachDB works unchanged -- point `DATABASE_URL` at it:

```
postgres://root@localhost:26257/space?sslmode=disable
```

2. Configure

```
cp .env.example .env
```

Minimum required: `DATABASE_URL`. Recommended for first run: `ADMIN_EMAIL`, `ADMIN_PASSWORD`, and `TMDB_API_KEY` / `OMDB_API_KEY` (free keys) so metadata import works.

3. Run

```
cargo run
```

Migrations apply automatically. The bootstrap admin is created if `ADMIN_EMAIL`/`ADMIN_PASSWORD` are set.

4. First login

```
curl -s -X POST localhost:8080/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"identifier":"admin@space.local","password":"admin12345"}'
```

Save `access_token` from the response; use it as `Authorization: Bearer ` on every call. Tokens expire in 15 min -- call `/api/auth/refresh` with your `refresh_token` to rotate them.

5. Add content

Either import metadata from a provider:

```
curl -X POST localhost:8080/api/metadata/import \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"source":"anilist","external_id":"16498"}'
```

...or scan a media folder (creates catalog skeletons matched by filename):

```
echo 'LIBRARY_ROOTS=/path/to/media' >> .env   # then restart, or:
curl -X POST localhost:8080/api/admin/library/scan \
  -H "Authorization: Bearer $TOKEN"
```

Probe files to extract tracks/durations:

```
curl -X POST localhost:8080/api/admin/files/probe-all \
  -H "Authorization: Bearer $TOKEN"
```

Transcode one file to HLS:

```
curl -X POST localhost:8080/api/admin/files/<FILE_ID>/transcode \
  -H "Authorization: Bearer $TOKEN"
```

Then play it in the web client (`/`) or hit

`GET /api/files//master.m3u8` directly with any HLS player.

6. Enable downloads for a user

```
curl -X POST localhost:8080/api/admin/users/<USER_ID>/permissions \
  -H "Authorization: Bearer $ADMIN_TOKEN" -H 'Content-Type: application/json' \
  -d '{"permission":"download"}'
```

The user can now mint signed links:

`POST /api/files/{file_id}/download-url {"device_id": "...", "ttl_secs": 1800}`.

Troubleshooting

Symptom	Fix
`missing required environment variable: DATABASE_URL`	`.env` missing/not readable
Playback 409 "file not matched"	run library scan; file has no linked media yet
Transcode 503	ffmpeg not installed or wrong `FFMPEG_BIN`
Search returns nothing after fresh CRDB install	expected: FTS extras skipped; ILIKE fallback active

Architecture

A modular monolith: one binary, strict module boundaries, Postgres as the source of truth, Redis optional. Chosen over microservices for operational simplicity at self-host scale; module seams make later extraction easy.

```
main -? telemetry -? config -? db (migrate) -? AppState::build -? router
```

Module map

Module	Responsibility	Key files
`auth`	sessions, tokens, lockout, resets, permissions,	service.rs, middleware.rs
`users` + `settings`	profiles, admin directory, per-user settings blob	repo.rs
`catalog`	media/seasons/episodes/genres/people/studios/credits,	repo.rs, metadata/*
`search`	FTS+trigram w/ ILIKE fallback, query parser, trending,	parser.rs, repo.rs
`tracking`	watch history, continue-watching, statuses, ratings,	repo.rs
`streaming`	storage trait, scanner, ffprobe, range responses,	storage.rs, scanner.rs, download.rs
`playback`	sessions, HLS transcode worker, thumbnails,	transcode.rs, markers.rs
`chat`	conversations/messages + the WS hub & gateway	hub.rs, ws.rs
`social`	follows, reviews, comments, reactions	repo.rs
`parties`	canonical playback state, event log, sync math	manager.rs, repo.rs
`intelligence`	taste DNA, vibe engine, home rails, smart collections	service.rs
`recommendations`	affinity recs + similar with Redis/memory caching	service.rs, cache.rs
`platform`	API keys, devices, notifications	repo.rs
`plugins`	runtime plugin registry + HTTP adapter	http_provider.rs
`ratelimit`	fixed-window limiter (Redis->memory fallback)	middleware.rs
`telemetry`/`router`/`docs`	logging+metrics init, route assembly, OpenAPI	--

Request lifecycle

1. TCP -> axum; TraceLayer logs span
2. Rate-limit middleware (per-IP buckets, auth paths stricter)
3. Metrics middleware records duration/route/status after handler
4. Extractors resolve auth (`CurrentUser` from Bearer / cookie / `X-API-Key`)
5. Handler -> repo layer (sqlx, parameterized) -> JSON or `ApiError`
6. `ApiError -> ProblemJson` uniform error body

Stateful pieces

- **AppState** holds pool, config, storage impl, WS hub, party manager, trans code queu e+wo rker, meta

data registry (`RwLock`, hot-swappable), cache, limiter, recommendation service, cancellation token.

- **Background tasks**: transcode workers (mpsc queue, kill-on-drop children), maintenance loop every 10 min (popularity recompute, expired token cleanup, stale session sweep), optional scanner scheduler.
- **Graceful shutdown**: SIGTERM/Ctrl-C cancels the tree, kills ffmpeg children, closes the pool.

Key design decisions

- **Opaque hashed tokens** (access+refresh) instead of JWTs: instant revocation, rotation on refresh, nothing to leak in payloads.
- **Unified `media` table** for movies/series/books with type-specific columns

nullable -- keeps search/collections/tracking polymorphic.

- **Keyset cursors** on stable sorts (created_at/title/popularity/rating); offset cursors only where ranking is volatile (relevance).
- **Party state = DB row + wall-clock projection**; events are append-only so reconnects replay deterministically.
- **Plugins over HTTP, not dynamic loading**: process isolation, any language, no unsafe ABI; registry swaps live under a RwLock.
- **Dialect portability**: PG-only FTS extras applied opportunistically; query paths

degrade to ILIKE so CockroachDB works unchanged.

Extending it

New domain? Add `src//{models,repo,handlers,routes}.rs`, merge its router in `router.rs`, add migrations -- everything else (auth, errors, pagination, metrics, docs skeleton) is already there.

Configuration Reference

All configuration is environment-based. Put it in `.env` (git-ignored) or the process environment; `.env.example` documents every key.

Server

Variable	Default	Description
`APP_HOST`	`0.0.0.0`	Bind address
`APP_PORT`	`8080`	HTTP port
`LOG_LEVEL`	`info`	tracing filter (`debug`, `space=trace`, ...)
`LOG_FORMAT` / `LOG_JSON`	`text`	`json` or `LOG_JSON=true` for structured logs
`METRICS_ENABLED`	`true`	Prometheus recorder at `/metrics`
`CORS_ORIGINS`	*(empty = allow all)*	comma-separated origins, e.g. `https://app.example.com`
`TRUST_PROXY`	`false`	honor `X-Forwarded-For` for client IPs (set behind nginx)

Database & cache

Variable	Default	Description
`DATABASE_URL`	**required**	Postgres DSN. CockroachDB DSN works unchanged.
`DB_MAX_CONNECTIONS`	`10`	pool size; scale with vCPU × 2-4
`REDIS_URL`	*(unset)*	enables Redis-backed cache + rate limiting. Unset/unreachabl

Auth & security

Variable	Default	Description
`ACCESS_TOKEN_TTL_SECS`	`900`	session token lifetime
`REFRESH_TOKEN_TTL_SECS`	`2592000`	refresh lifetime (30 d)
`SESSION_COOKIE_NAME`	`space_session`	HttpOnly cookie holding the access token
`COOKIE_SECURE`	`false`	set `true` when serving HTTPS
`PASSWORD_MIN_LENGTH`	`8`	also enforces letter+digit mix at this length
`LOCKOUT_THRESHOLD`	`5`	failed logins before lockout (per identity+IP)
`LOCKOUT_WINDOW_SECS`	`900`	failure counting window
`LOCKOUT_DURATION_SECS`	`900`	lockout duration after threshold
`RESET_LINK_BASE`	`http://localhost:8080/reset`	prefix used in reset links

Admin bootstrap

Variable	Description
`ADMIN_EMAIL`, `ADMIN_PASSWORD`	created on startup if absent; role=admin

Media & storage

Variable	Default	Description
`LIBRARY_ROOTS`	*(unset)*	comma-separated absolute paths to scan
`STORAGE_DIR`	`./data/storage`	transcode output, thumbnails, extracted subs
`SCANNER_INTERVAL_SECS`	`0` (off)	background rescan period
`FFMPEG_BIN` / `FFPROBE_BIN`	`ffmpeg`/`ffprobe`	binary paths
`TRANSCODE_HEIGHTS`	`1080,720,480`	HLS ladder (source-capped)

Metadata providers & plugins

Variable	Description
`TMDB_API_KEY`	enables built-in `tmdb` source
`OMDB_API_KEY`	enables built-in `imdb` source (via OMDb)
`ANILIST_ENABLED`	default true; enables `anilist` source
`PLUGIN_ALLOW_HTTP`	reserved policy flag for plugin egress

Rate limiting

Variable	Default
`RATE_LIMIT_ENABLED`	`true`
`RATE_LIMIT_GLOBAL_PER_MIN`	`300` per IP
`RATE_LIMIT_AUTH_PER_MIN`	`20` on login/register/refresh/forgot

Notes

- Everything degrades gracefully: no Redis -> memory cache/limiter; no ffmpeg -> jobs fail explicitly while streaming keeps working; no provider keys -> those sources simply don't appear.
- CockroachDB: PG-only FTS extras are applied opportunistically and skipped on unsupport

ed engin es -- searc h falls back to ILIKE matc hing.

Admin security

Variable	Default	Description
ADMIN_LOCAL_ONLY	rue	/api/admin/* and /admin.html respond only to loopback peers.
EMAIL_DEV_ECHO	rue	returns verification codes in API responses for localhost te

Database

PostgreSQL dialect (CockroachDB wire-compatible; see portability notes).

Migrations live in `migrations/` and apply automatically on boot.

Migration	Domain
`0001_core`	users, user_permissions, sessions
`0002_catalog`	media (+FTS vector), seasons, episodes, genres, people, stud
`0003_tracking`	watch_history, list_entries, favorites, user_lists(+items)
`0004_streaming`	library_paths, media_files, media_streams, playback_sessions
`0005_social`	follows, reviews, comments, reactions, parties(+members/even
`0006_security_plugins`	user_settings, password_resets, login_lockouts, plugins
`0007_intelligence`	media_markers, playback_handoffs, party_reactions, smart_col
`0008_platform`	book columns on media, reading_progress, api_keys, devices,
`0009_download_permission`	extends permission CHECK with `download`

Entity relationships (core)

```
users -?-< sessions
       +-< list_entries >- media        media -?-< seasons -< episodes
       +-< watch_history >- media              +-< media_genres >- genres
       +-< reviews      >- media               +-< media_studios >- studios
       +-< favorites    >- media               +-< credits >- people
       +-< reading_progress                    +-< external_mappings
       +-< api_keys   +-< devices
       +-< notifications

library_paths -< media_files -< media_streams
             media_files -< playback_sessions, transcode_jobs, thumbnails,
                            media_markers, download_grants

parties -< party_members / party_events / party_reactions
conversations -< conversation_members, messages -< message_receipts
smart_collections (rules JSONB -> executed against media at read time)
```

Conventions

- **UUID v7 primary keys** -- time-ordered, index-friendly, cursor-safe
- **Enums as TEXT + CHECK** -- portable across PG-wire databases; mapped in Rust via `#[derive(sqlx::Type)] #[sqlx(rename_all =

"snake_case")]`

- **Tokens/secrets stored hashed** (SHA-256): session tokens, refresh tokens, reset tokens, API-key secrets, download-signing secret (server-only)
- Watch history uniqueness via functional unique index on `(user_id, media_id, COALESCE(episode_id, nil))`; upserts handled app-side with `FOR UPDATE` to stay portable
- Party sync state is denormalized onto the `parties` row (`is_playing`, `position_seconds`, `state_updated_at`, `last_event_seq`

) with an append-only `party_events` log for reconnect replay

Portability: CockroachDB & TiDB

- **CockroachDB**: point `DATABASE_URL` at it. On first migrate, the optional layer (`migrations-optional/pg_search.sql`) applies statement-by-statement; unsupported statements (pg_trgm) are skipped with one info log. FTS queries detect the missing vector and fall back to ILIKE automatically.
- **TiDB**: speaks MySQL wire protocol -- requires swapping sqlx driver types. Not recommended until a dedicated milestone; CRDB delivers distributed SQL with zero application changes.

Ops cheat-sheet

```
-- biggest libraries by storage
SELECT lp.root_path, pg_size_pretty(SUM(f.file_size)::bigint)
FROM media_files f JOIN library_paths lp ON lp.id=f.library_path_id
GROUP BY 1 ORDER BY 2 DESC;

-- most watched (14d)
SELECT m.title, COUNT(*) FROM watch_history wh
JOIN media m ON m.id=wh.media_id
WHERE wh.updated_at > now()-interval '14 days' GROUP BY 1 ORDER BY 2 DESC LIMIT 10;

-- stuck transcodes
UPDATE transcode_jobs SET status='failed', error='stuck'
WHERE status='running' AND started_at < now()-interval '6 hours';
```

Deployment

1. Production stack (Docker)

```
cp .env.example .env    # set POSTGRES_PASSWORD, ADMIN_*, CORS_ORIGINS, provider keys
docker compose -f docker-compose.prod.yml up -d --build
```

Services: `app` (this image) · `postgres:16` · `redis:7` (AOF persistence) ·

`nginx` edge proxy. Media and storage live in named volumes

(`media`, `storage`); Postgres data in `pgdata`.

2. TLS

`ops/nginx.conf` ships HTTP-ready with a commented 443 server block:

```
mkdir -p ops/certs && cp fullchain.pem privkey.pem ops/certs/
# uncomment the TLS server block, then:
docker compose -f docker-compose.prod.yml restart proxy
```

For public deployments prefer certbot/Let's Encrypt mounted into the same path.

Set `TRUST_PROXY=true` and `COOKIE_SECURE=true` in `.env` when terminating TLS.

3. Reverse-proxy behavior (already configured)

- `/ws` -> upgrade headers + 1 h read timeout
- `/api/(files|hls|transcodes)` -> `proxy_buffering off`, Range passthrough, long read timeouts (streaming-friendly)
- `/api/auth/login` -> edge rate limit (`10r/m` burst 5), defense-in-depth on top of the app limiter
- Everything else -> standard proxy headers

4. Backups & recovery

Nightly logical backup + 14-day retention + gzip verification:

```
./ops/backup.sh                    # BACKUP_DIR/CONTAINER overridable
```

Restore **rehearsal** (do this quarterly -- an untested backup is a hope):

```
./ops/restore.sh ops/backups/space-20260101T000000Z.sql.gz
curl -fsS localhost:8080/api/ready   # must be {"ready":true,...}
```

Point-in-time recovery: enable Postgres WAL archiving
(`archive_command = 'cp %p /backups/wal/%f'`) and use `pgBackRest`/`wal-g`;
the restore script's flow stays the same.

Media files are outside the DB by design -- snapshot the `media`/`storage`
volumes (or your object store bucket) alongside DB dumps.

5. Monitoring

Signal	Source
Liveness	`GET /health`
Readiness	`GET /api/ready` (DB ping; wire to LB health checks)
Metrics	`GET /metrics` -- HTTP latency histogram, request counters,
Business stats	`GET /api/stats`

Scrape example (prometheus.yml):

```
scrape_configs:
  - job_name: space
    static_configs: [{ targets: ["app:8080"] }]
```

Suggested alerts: p95 latency > 500 ms for 5 m · error-rate > 2% ·
`rate_limit_rejected_total` spike · transcode failure ratio > 20% ·
`up == 0` on readiness.

6. Scaling notes

- App is stateless except WS presence -- run N replicas behind nginx (`ip_hash` or sticky cookies) until Redis pub/sub fan-out lands (ROADMAP M9)

- Postgres: raise `DB_MAX_CONNECTIONS` with pooler (PgBouncer) in front; read replicas work as-is for search-heavy traffic
- Transcodes are CPU-bound: scale workers horizontally by running extra app instances pointed at a shared `STORAGE_DIR` (NFS/object storage), or pin jobs per node
- HLS segments are immutable -> put a CDN in front of `/api/hls/*` and `/api/files/*/master.m3u8` with auth-aware caching rules

7. Zero-downtime upgrades

Migrations are additive-first; deploys follow blue/green:
start new version -> wait `/api/ready` -> shift proxy weight -> drain old
(with_graceful_shutdown drains in-flight requests and WS upgrades).

8. Private origin behind a CDN (media protection)

Goal: the internet talks to your edge; only the edge talks to SPACE.

```
Internet --? CDN / reverse proxy (TLS, caching)
                 ?  injects X-Origin-Token: <ORIGIN_INTERNAL_TOKEN>
                 ?
             SPACE origin (not publicly routable)
```

1. Set a long random `ORIGIN_INTERNAL_TOKEN` on the origin.
2. Configure the proxy to add the header on `/api/playback/keys/*` and `/api/dl/*` (nginx):

```
location ~ ^/api/(playback/keys|dl)/ {
    proxy_set_header X-Origin-Token "the-long-random-token";
    proxy_pass http://space-origin:8080;
}
```

Direct hits without the header get `403 origin token required`. Cache media at the edge but never cache the key endpoint (`Cache-Control: no-store` is set automatically for it).

3. Optional hardening: `HLS_ENCRYPTION=true` encrypts transcode outputs with AES-128; segment keys are released only against valid short-lived playback grants (`POST /api/playback/session`, then pass the grant to `GET /api/playback/keys/{file_id}?grant=...`). Rotate or revoke per-media keys via the admin security endpoints ? revocation stops future decryption even for copies already downloaded in pieces.
4. Every `POST /api/resolve` URL carries `wm=` ? a forensic watermark binding user +

resolution + hour. Keep request logs at the edge and a leaked copy's manifest traces back to one session.

SPACE -- Complete API Reference

***Base URL:** `http://localhost:8080`*

***Interactive docs:** `GET /docs` (Swagger UI) · `GET /api-docs/openapi.json` (OpenAPI spec)*

***Version header:** Every response includes `X-API-Version: space/`*

***v1 compatibility:** Legacy `/v1/*` paths are auto-rewritten to `/api/*` with deprecation tag*

Table of Contents

- Authentication
- Error Format
- Pagination
- Rate Limiting
- Auth & Identity
- User Platform
- Admin Fortress
- Catalog & Metadata
- Search & Discovery
- Sources & Plugins
- Resolution & Playback
- Playback Security
- Streaming & Protection
- Tracking & Progress
- Social & Chat
- Watch Parties
- Moderation & Safety
- Intelligence
- Media Graph
- Realtime (WebSocket)
- Card Economy
- Library & Playlists
- File Management
- Ops & Observability
- Developer Experience

Authentication

SPACE supports three authentication methods:

Bearer Token (primary)

```
# Login to get tokens
curl -X POST /api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"identifier":"admin","password":"secret"}'
```

```
# -> { "access_token": "...", "refresh_token": "...", "user": {...} }

# Use the access token
curl -H "Authorization: Bearer <access_token>" /api/users/me
```

API Keys (machine-to-machine)

```
# Create a scoped API key
curl -X POST /api/me/keys -H "Authorization: Bearer $TOKEN" \
  -d '{"name":"ci-pipeline","scopes":["read","downloads"]}'

# Use it
curl -H "X-API-Key: sk_..." /api/search?q=test
```

Scopes: `read` (GET), `write` (mutations), `downloads` (file access).

Session Cookies (browser)

HttpOnly, SameSite=Lax cookies are set on login/refresh. Works with the SPA.

Token lifecycle:

- Access token: 15 min (configurable via `ACCESS_TOKEN_TTL_SECS`)
- Refresh token: 30 days, rotating on use, hashed at rest
- Single-use password reset tokens: 30 min TTL
- All tokens are SHA-256 hashed in the database

Error Format

Every error returns a uniform `ProblemJson` body:

```
{
  "code": "validation_error",
  "message": "email is required",
  "trace_id": "a1b2c3d4e5f6"
}
```

Status	`code`	Meaning
400	`validation_error`	Bad request / missing fields
401	`unauthorized`	No auth or invalid token
403	`forbidden`	Insufficient permissions
404	`not_found`	Resource doesn't exist
409	`conflict`	Duplicate resource
429	`rate_limited`	Too many requests (includes `Retry-After` header)
503	`unavailable`	Service temporarily down
500	`internal_error`	Server bug

Pagination

Most list endpoints use cursor-based pagination:

```
GET /api/media?limit=20&cursor=<opaque>
```

- `limit`: items per page (clamped 1-100)
- `cursor`: opaque string from `next_cursor` in response; pass it back as `cursor=`
- Response always includes `next_cursor: null | "..."` when exhausted

Rate Limiting

Scope	Limit	Window
Global (per-IP)	300 req/min	sliding
Auth endpoints	20 req/min	sliding
Login (nginx edge)	10 req/min burst 5	fixed

Returns `429` with `Retry-After: 60` header. Redis-backed when available, falls back to in-memory.

Auth & Identity

POST /api/auth/register

Create a new account. Automatically grants a 7-day trial access card.

```
curl -X POST /api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","email":"alice@example.com","password":"Secure123!"}'
```

Body: `{ username, email, password }`

Response: `{ access_token, refresh_token, user: { id, username, email, role } }`

POST /api/auth/login

Login with username or email. Supports TOTP 2FA for admins.

```
curl -X POST /api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"identifier":"alice","password":"Secure123!"}'
# Admin with 2FA:
# -d '{"identifier":"admin","password":"...","totp_code":"123456"}'
```

Body: `{ identifier, password, totp_code? }`

Response: `{ access_token, refresh_token, user }`

Lockout: 5 failed attempts per (identity, IP) -> 15-minute lockout

---

POST /api/auth/refresh

Rotate to a new token pair. Old refresh token is invalidated.

```
curl -X POST /api/auth/refresh \
  -H "Content-Type: application/json" \
  -d '{"refresh_token":"..."}'
```

POST /api/auth/logout

Invalidate the current session.

POST /api/auth/logout-all

Invalidate every session for the user.

POST /api/auth/password/forgot

Send a password reset email. Enumeration-safe (always returns OK).

POST /api/auth/password/reset

Consume the reset token and set a new password.

```
curl -X POST /api/auth/password/reset \
  -H "Content-Type: application/json" \
  -d '{"token":"...","new_password":"NewSecure123!"}'
```

POST /api/users/me/password

Change password for the authenticated user (requires current password).

```
curl -X POST /api/users/me/password \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"current_password":"old","new_password":"new"}'
```

POST /api/auth/email/request-code

Request an email verification code. Hashed at rest, expires in 15 min, locks after 5 bad tries.

POST /api/auth/email/verify

Verify the code. In dev mode (`EMAIL_DEV_ECHO=true`), the code is echoed in the API response.

GET /api/auth/sessions

List all active sessions for the current user.

DELETE /api/auth/sessions/{session_id}

Revoke a specific session.

POST /api/me/keys

Create a scoped API key.

```
curl -X POST /api/me/keys -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"my-script","scopes":["read"]}'
```

Response: `{ key_id, key: "sk_...", name, scopes, created_at }` -- the `key` is shown only once.

GET /api/me/keys

List all API keys (secrets masked).

DELETE /api/me/keys/{key_id}

Revoke an API key.

POST /api/admin/users/{user_id}/permissions

Grant a fine-grained permission.

```
curl -X POST /api/admin/users/$UID/permissions \
  -H "Authorization: Bearer $ADMIN" \
  -H "Content-Type: application/json" \
  -d '{"permission":"download"}'
```

Valid permissions: `download`, `manage_library`, `moderate_content`

GET /api/admin/users/{user_id}/permissions

List permissions for a user.

DELETE /api/admin/users/{user_id}/permissions/{permission}

Revoke a permission.

GET /api/users/{username}

Public profile for any user.

GET /api/users/me

Current user's profile.

PATCH /api/users/me

Update current user's profile.

User Platform

GET|PUT|PATCH /api/users/me/settings

Per-user preferences (playback, UI, notifications). Supports full replace (PUT) or partial patch (PATCH).

POST /api/me/devices

Register a client device.

```
curl -X POST /api/me/devices -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"Living Room TV","platform":"android-tv"}'
```

GET /api/me/devices

List registered devices.

DELETE /api/me/devices/{device_id}

Revoke a device (unbinds from download links).

GET /api/me/notifications

Paginated notification feed.

GET /api/me/notifications/unread-count

Returns `{ "count": 3 }`.

POST /api/me/notifications/{notification_id}/read

Mark a notification as read.

GET /api/me/search-history

List recent search queries.

DELETE /api/me/search-history

Clear search history.

GET /api/me/behavior

Personal consumption analytics: peak hours, preferred genres, device usage.

PUT /api/me/presence

Update your online status.

GET /api/presence?ids=uuid1,uuid2

Batch presence lookup.

Admin Fortress

> ***Security:** All `/api/admin/*` endpoints are localhost-only by default (`ADMIN_LOCAL_ONLY=true`). Set to `false` to allow remote access.*

GET|POST /api/admin/security/lockdown

Toggle site-wide lockdown (503 on all public routes). Customizable message.

GET /api/admin/audit-logs?action=&limit=

Filterable audit trail of all sensitive actions with actor IP and timestamp.

GET /api/admin/users

Paginated user listing (admin-only).

GET|POST /api/admin/debug/dump-secrets

Honeypot trap. Any touch = permanent ban + session wipe + watchdog trip.

POST /api/admin/security/totp/setup

Generate TOTP provisioning URI for an admin.

POST /api/admin/security/totp/enable

Enable TOTP 2FA for an admin.

POST /api/admin/security/totp/disable

Disable TOTP 2FA.

GET|PATCH /api/admin/security/policy

View and tune the playback risk engine thresholds.

GET /api/admin/security/events

Decision audit log: every challenge/revoke with signals snapshot.

GET /api/admin/security/grants

List all active playback grants.

POST /api/admin/security/grants/{id}/revoke

Revoke a single grant.

POST /api/admin/security/users/{user_id}/revoke-grants

Revoke all grants for a user.

POST /api/admin/security/keys/{media_ref}/rotate

Rotate encryption keys for a media item.

POST /api/admin/security/keys/{media_ref}/revoke

Irreversibly revoke all keys for a media item.

Catalog & Metadata

GET /api/media

List media with filters.

```
GET /api/media?media_type=movie&sort=rating&limit=20&cursor=...
```

Filters: `media_type`, `genre`, `year_from`, `year_to`, `sort` (title/rating/popularity/created_at).

POST /api/media

Create a media entry (admin).

GET /api/media/{id}

Get full media detail with genres, studios, credits, external mappings.

PATCH /api/media/{id}

Update a media entry (admin).

DELETE /api/media/{id}

Delete a media entry (admin).

GET /api/media/{id}/episodes

List episodes for a series.

POST /api/admin/media/{id}/seasons

Create a season (admin).

POST /api/admin/media/{id}/seasons/{season_id}/episodes

Create an episode (admin).

GET /api/metadata/search

Search external metadata providers.

```
GET /api/metadata/search?q=cowboy+bebop&provider=tmdb
```

GET /api/metadata/sources

List available metadata providers (built-in + plugins).

POST /api/metadata/import

Import metadata from an external provider.

```
POST /api/metadata/import
{ "source": "tmdb", "external_id": "1234" }
# -> Full MediaDetail with imported data
```

POST /api/admin/metadata/refresh

Trigger metadata refresh for stale external mappings.

GET /api/admin/quality/conflicts

Review metadata conflicts from multiple sources.

GET /api/admin/quality/duplicates

Find potential duplicate catalog entries.

POST /api/admin/quality/duplicates/auto-resolve

Merge weaker duplicates into the stronger entry.

GET /api/admin/storage/analytics

Disk usage breakdown per media type.

Search & Discovery

GET /api/search?q=

Full-text search with tsvector ranking (ILIKE fallback on CockroachDB).

```
GET /api/search?q=action+2020-2025+1080p&limit=20
```

GET /api/search/parse?q=

Parse a natural-language query into structured filters.

```
GET /api/search/parse?q=action 2020-2025 1080p english
# -> { genres: ["action"], year_from: 2020, year_to: 2025, quality: "1080p", language: "english" }
```

GET /api/search/all?q=

Unified cross-source search: parallel fan-out across local catalog + every healthy source.

GET /api/search/suggest?q=

Type-ahead suggestions (fast, truncated).

GET /api/discover

Curated discovery feed with filter parameters.

GET /api/trending?window=week

Windowed popularity from real watch activity. Windows: `day`, `week`, `month`.

GET /api/media/{id}/similar

Similar media based on shared genres + studios, cached per item.

GET /api/media/{id}/similar-scores

Detailed similarity scores.

GET /api/intelligence/vibe?q=

Natural-language mood matching ("cozy feel-good") over genres, keywords, and pacing.

GET /api/completed?type=movie

Finished/released catalog filtered by type.

GET /api/toplists?kind=movies|series|anime|books|all

Curated top charts. Local popularity first, TMDB top-rated fallback. 30-min cache.

GET /api/random?type=&min_rating=

Random pick with type and minimum-rating filters.

GET /api/upcoming

Episodes airing in the next 14 days.

POST /api/media/{id}/availability

Where can I play this? Matches across every healthy source + counts local files.

Sources & Plugins

Built-in Sources

Source	Category	Content
ia-films	movies	Internet Archive public-domain films
ia-audio	audiobooks	IA audio collections
gutenberg	books	75k+ Project Gutenberg ebooks
mangadex	manga	Licensed manga with chapters/pages
audius	music	Artist-uploaded music tracks
librivox	audiobooks	Free public-domain audiobooks

GET /api/sources

List all registered sources.

GET /api/sources/status

Per-source health: latency, success rates, circuit state.

GET /api/sources/rankings

Head-to-head benchmark scores (daily worker).

GET /api/sources/{name}/browse

Browse trending/top rails for a source.

POST /api/admin/sources/{name}/sync

Import source items into the local catalog.

POST /api/admin/sources/register-url

Register an HTTP source by base URL. Probes `manifest.json -> /items -> /search`.

Source Debug (admin)

```
GET /api/admin/sources/{name}/debug-search?q=...
GET /api/admin/sources/{name}/debug-item/{external_id}
```

Plugin Management

```
# List plugins
GET /api/admin/plugins

# Install a plugin
POST /api/admin/plugins
{ "name": "my-metadata", "kind": "metadata", "base_url": "https://...", "enabled": true }

# Update / Delete
PATCH /api/admin/plugins/{id}
DELETE /api/admin/plugins/{id}

# Test a plugin
POST /api/admin/plugins/{id}/test

# Sync source plugin into catalog
POST /api/admin/plugins/{id}/sync
```

GET /api/metadata/sources

List all enabled metadata providers (built-ins + plugins).

Web Sources (Galaxy Engine)

Self-learning web scrapers. Paste a site URL and SPACE learns its structure.

```
# Create
POST /api/admin/websources
{ "site": "https://movies.example", "category": "movie" }

# List / Update / Delete
GET /api/admin/websources
PUT /api/admin/websources/{id}
DELETE /api/admin/websources/{id}

# Test scraping
POST /api/admin/websources/{id}/test

# Re-learn after site redesign
POST /api/admin/websources/{id}/relearn

# View learned recipe
GET /api/admin/websources/{id}/recipe
```

Hub Federation

Connect SPACE instances as fallback providers.

```
# Manage hubs
GET /api/admin/hubs
POST /api/admin/hubs
{ "url": "http://192.168.1.50:8080" }
PUT /api/admin/hubs/{id}
DELETE /api/admin/hubs/{id}

# Sync recipes
POST /api/admin/hubs/{id}/pull
POST /api/admin/hubs/{id}/push

# Test connectivity
POST /api/admin/hubs/{id}/test
```

Hub Public API (server-side)

```
GET /api/hub/find?q=...        # Search across hub
GET /api/hub/item?id=...       # Get item details
GET /api/hub/sources           # List hub's sources
POST /api/hub/sources          # Upload source recipe
```

Requires `SOURCE_HUB_TOKEN` header on the hub server.

Stremio Extensions

Install community addons by manifest URL. Catalogs join unified search; streams join playback resolution.

```
# Install
POST /api/admin/extensions
{ "url": "https://torrentio.strem.fun/manifest.json" }

# Manage
GET /api/admin/extensions
PATCH /api/admin/extensions/{id}
DELETE /api/admin/extensions/{id}

# Discover community addons
GET /api/admin/extensions/discover

# Raw resource access (1:1 Stremio SDK)
GET /api/extensions/{id}/catalog/{type}/{catalog_id}
GET /api/extensions/{id}/stream/{type}/{id}
GET /api/extensions/{id}/meta/{type}/{id}
GET /api/extensions/subtitles/{type}/{id}
```

Resolution & Playback

POST /api/resolve

Central resolver pipeline (11 stages). The primary way to get a playable stream.

```
POST /api/resolve
{
  "title": "Cowboy Bebop",
  "media_type": "anime",
  "year": 1998,
  "preferences": {
    "quality": "1080p",
    "language": "japanese",
    "subtitles": "english"
  }
}
```

Response:

```
{
  "stream_url": "https://...",
  "quality": "1080p",
  "source": "miruro",
  "format": "hls",
  "fallbacks": [...],
  "watermark": "a1b2c3d4e5f6g7h8"
}
```

Every URL carries `wm=` forensic watermark binding user + session + hour.

POST /api/playback/resolve

Legacy resolve endpoint (preserved for existing clients).

POST /api/playback/lookup

Availability-first resolver: returns where a title can be played without triggering full resolve.

```
POST /api/playback/lookup
{ "title": "Arrival", "media_type": "movie" }
# -> { "available": true, "sources": [...], "local_files": 2 }
```

POST /api/playback/manga/chapters

List chapters for a manga title.

POST /api/playback/manga/pages

Get page images for a chapter.

POST /api/playback/manga/open

One-shot open: detail + chapters + first pages.

GET /api/playback/resume/{media_id}

Resume info for a media item (best file, stored position, track preferences).

GET /api/playback/resume/episode/{episode_id}

Resume info for a specific episode.

GET /api/playback/next-hint

Suggests the next episode or related media.

POST /api/playback/handoff

Create a one-time cross-device handoff token.

POST /api/playback/handoff/redeem

Redeem a handoff token on another device to resume playback.

Playback Security

POST /api/playback/session

Authenticate -> risk gate -> short-lived signed grant. Entry point for all playback.

```
POST /api/playback/session
{ "media_id": "...", "file_id": "...", "client_fingerprint": "..." }
# -> { "grant": "eyJ...", "expires_in": 600 }
```

POST /api/playback/grant/segment

Mint a child grant scoped to a specific HLS segment (<=120s TTL).

POST /api/playback/offline-keys

Issue an offline playback license (24h TTL, bound to user+media+device).

GET /api/playback/offline-keys

List your offline licenses.

GET /api/playback/offline-keys/{license_id}/validate

Validate an offline license.

DELETE /api/playback/offline-keys/{license_id}

Revoke an offline license.

Streaming & Protection

GET /api/files/{id}/raw

HTTP Range streaming for seeking. Supports `Range: bytes=0-1023` headers.

GET /api/files/{id}/subtitles

Extracted + converted subtitles (SRT/ASS -> VTT).

POST /api/files/{id}/download-url

Generate a signed download URL (HMAC-SHA256, device-bound, expiring).

```
POST /api/files/{id}/download-url
{ "device_id": "..." }
# -> { "url": "https://.../dl/eyJ...", "expires_at": "..." }
```

GET /api/dl/{token}

Redeem a signed download URL. Returns the file with `Content-Disposition: attachment`.

POST /api/admin/files/{id}/probe

On-demand FFprobe of a file.

POST /api/admin/files/probe-all

Probe all unprobed files.

POST /api/admin/library/scan

Scan configured `LIBRARY_ROOTS` for new media.

GET /api/lyrics?q=

Search for synchronized lyrics by query.

GET /api/lyrics/providers

List available lyrics providers.

GET /api/media/{id}/lyrics

Get lyrics for a specific media item.

Tracking & Progress

PUT /api/tracking/progress

Report video progress (auto-completes at 92%).

```
PUT /api/tracking/progress
{ "media_id": "...", "position_seconds": 1234, "duration_seconds": 5400 }
```

PUT /api/tracking/reading

Report reading progress for books/manga.

```
PUT /api/tracking/reading
{ "media_id": "...", "position_percent": 45.2, "page": 120 }
```

PUT /api/progress

Universal progress router (auto-routes by media_type).

GET /api/me/media/{id}/reading

Get reading progress for a media item.

GET /api/continue?types=movie,manga

Continue rail: merged watching + reading + listening progress, filterable by type.

GET /api/me/continue-watching

Continue watching (video only).

PUT /api/me/media/{id}/status

Set your list status for a media item.

```
PUT /api/me/media/{id}/status
{ "status": "watching" }
# Values: watching, completed, on_hold, dropped, planned
```

DELETE /api/me/media/{id}/status

Remove your status.

PUT /api/media/{id}/rating

Rate a media item (1-10).

DELETE /api/media/{id}/rating

Remove your rating.

POST /api/media/{id}/favorite

Toggle favorite.

GET /api/me/history

Paginated watch/reading history (cursor-based).

GET /api/me/history/settings

Get history display/filter settings.

PUT /api/me/history/settings

Update history settings.

DELETE /api/me/history

Wipe entire watch history. Audited.

GET /api/me/ratings

List your ratings.

GET /api/me/favorites

List your favorites.

GET /api/me/list

Get your combined list (all statuses).

POST /api/me/lists

Create a custom named list.

GET /api/me/lists

List your custom lists.

GET /api/lists/{id}/items

List items in a custom list.

POST /api/lists/{id}/items/{media_id}

Add an item to a list.

DELETE /api/lists/{id}/items/{media_id}

Remove an item from a list.

GET /api/me/summary

Dashboard summary: watch hours, counts, reading progress, notifications, card status.

GET /api/me/export

Full personal-data export as JSON.

Playback Session Tracking

```
POST /api/playback/sessions         # Create a session
GET /api/playback/sessions           # List my sessions
POST /api/playback/sessions/{id}/heartbeat  # Keep alive
DELETE /api/playback/sessions/{id}   # End session
```

Social & Chat

Reviews

```
GET /api/media/{id}/reviews          # List reviews for media
POST /api/media/{id}/reviews         # Create a review
DELETE /api/reviews/{id}             # Delete your review
```

Comments & Reactions

```
GET /api/comments?media_id=...       # List comments
POST /api/comments                   # Create a comment
DELETE /api/comments/{id}            # Delete your comment
POST /api/reactions                  # Toggle a reaction
GET /api/reactions?media_id=...      # Get reaction summary
```

Follows & Activity Feed

```
POST /api/users/{id}/follow          # Follow a user
GET /api/me/following                # Who you follow
GET /api/me/followers                # Who follows you
GET /api/social/feed                 # Activity feed (own + followed)
```

Conversations & DMs

```
GET /api/chat/conversations                  # List conversations
POST /api/chat/conversations                 # Create a conversation
GET /api/chat/conversations/{id}/messages    # List messages
POST /api/chat/conversations/{id}/messages    # Send a message
POST /api/chat/conversations/{id}/members/{user_id}  # Add member
DELETE /api/chat/conversations/{id}/members/me        # Leave
POST /api/chat/conversations/{id}/read        # Mark read
```

Message Edit & Delete

```
PATCH /api/chat/messages/{id}        # Edit (adds edited_at marker)
DELETE /api/chat/messages/{id}       # Soft-delete (keeps audit)
```

Friends System

```
POST /api/friends/{id}/request       # Send friend request
DELETE /api/friends/{id}             # Remove friend
POST /api/friends/requests/{id}/accept   # Accept request
POST /api/friends/requests/{id}/decline  # Decline request
DELETE /api/friends/requests/{id}    # Cancel outgoing request
GET /api/friends/requests            # List pending requests
GET /api/friends                     # List friends
```

Media Sharing

```
POST /api/share/media                # Share media with a user
GET /api/me/shares/incoming          # Incoming shares
GET /api/me/shares/history           # Share history
POST /api/shares/{id}/viewed         # Mark as viewed
DELETE /api/shares/{id}              # Delete a share
```

Watch Parties

Party Lifecycle

```
POST /api/parties                    # Create a party
POST /api/parties/join-by-code       # Join by short code
POST /api/parties/{id}/join          # Join by ID
POST /api/parties/{id}/leave         # Leave
DELETE /api/parties/{id}             # End (host only)
GET /api/me/parties                  # List your parties
GET /api/parties/{id}                # Party details
GET /api/parties/{id}/members        # List members
```

Host Controls

```
POST /api/parties/{id}/control       # Play/pause/seek commands
GET /api/parties/{id}/state          # Current sync state
GET /api/parties/{id}/events         # Event history (reconnect replay)
POST /api/parties/{id}/latency       # Report latency for drift correction
```

Reactions & Transfers

```
POST /api/parties/{id}/reactions         # Add emoji reaction
GET /api/parties/{id}/reactions/heatmap  # Aggregate heatmap
POST /api/parties/{id}/transfer          # Transfer host role
POST /api/parties/{id}/promote           # Promote to co-host
```

Moderation & Safety

User Reports

```
POST /api/reports                    # Submit a report
GET /api/admin/reports?status=       # List reports (admin)
PATCH /api/admin/reports/{id}        # Update status
POST /api/admin/reports/{id}/accept  # Accept report
POST /api/admin/reports/{id}/reject  # Reject report
```

Report targets: `media`, `metadata`, `comment`, `chat_message`, `party_user`, `user`.

Blocks & Mutes

```
POST /api/users/{id}/block           # Block a user
DELETE /api/users/{id}/block         # Unblock
GET /api/me/blocks                   # List blocked users
POST /api/users/{id}/mute            # Mute (optional duration)
DELETE /api/users/{id}/mute          # Unmute
```

User Appeals

```
POST /api/appeals                    # Submit an appeal
GET /api/me/appeals                  # Your appeals
GET /api/admin/appeals               # All appeals (admin)
POST /api/admin/appeals/{id}/accept  # Accept appeal
POST /api/admin/appeals/{id}/reject  # Reject appeal
```

Privacy Controls

```
GET /api/me/privacy                  # Get privacy settings
PATCH /api/me/privacy                # Update privacy settings
```

Intelligence

Taste Profile

```
GET /api/me/taste
# -> { genre_weights: {...}, favorite_decades: [...], avg_runtime: 120, anime_affinity: 0.3, completion_rate: 0.72 }
```

Home Rails

```
GET /api/home
# -> { continue: [...], trending: [...], because_you_watched: [...], new_episodes: [...], friends: [...], recommendations: [...], vibe: [...] }
```

Recommendations

```
GET /api/recommendations              # Personalized recommendations
GET /api/media/{id}/similar-scores    # Detailed similarity scores
```

Personal Media Twin

```
GET /api/me/twin
# -> Behavioral model: format preferences, genre weights, completion pace, creators, discovery ratio, current interests
```

Cross-Media Journeys

```
GET /api/discover/journeys
# -> Adaptation chains: book -> movie -> series -> remake...
```

AI Copilot

```
GET /api/admin/ai/config              # View AI configuration
PATCH /api/admin/ai/config            # Update AI configuration
POST /api/admin/ai/analyze            # Trigger analysis
POST /api/admin/ai/apply              # Apply suggested changes
GET /api/admin/ai/events              # AI action audit log
GET /api/admin/ai/status              # Engine status
```

Mood Catalog

```
GET /api/intelligence/moods           # List available moods
GET /api/intelligence/vibe?q=cozy     # Match media to mood
```

Smart Collections

```
POST /api/collections                 # Create a collection
GET /api/collections/{id}/items       # List items
DELETE /api/collections/{id}          # Delete collection
```

Heuristic Engine

```
GET /api/heuristic/status             # Engine status
GET /api/me/heuristic                 # Your behavioral profile
DELETE /api/me/heuristic              # Wipe your data
GET /api/me/heuristic/wrapped         # Year-in-review recap
GET /api/me/heuristic/insights        # Deep consumption insights
GET /api/me/heuristic/predictions     # Next-up predictions
GET /api/me/heuristic/similar/{id}    # Per-item similarity scores
```

Media Graph

Relationships

12 typed relationships: `adaptation_of`, `sequel_of`, `spin_off_of`, `same_universe`, `remake_of`, `prequel_of`, `parent_story`, `side_story`, `alternative_version`, `shared_character`, `crossover`, `inspired_by`.

```
POST /api/media/{id}/relationships        # Create a relationship
GET /api/media/{id}/relationships         # List relationships
DELETE /api/media/{id}/relationships/{to}/{rel}  # Remove
```

Traversal

```
GET /api/media/{id}/related      # One-hop neighborhood, grouped by relation
GET /api/media/{id}/journey      # BFS <=3 hops across all edges
```

Realtime (WebSocket)

Endpoint: `GET /ws?token=`

Connection Flow

1. Connect with a valid access token as query param
2. Receive `hello` frame with `user_id` and `online_users[]`
3. Subscribe to rooms: `subscribe` frame with `room` field

Rooms

Room Pattern	Purpose	Auth
`__global__`	Presence changes, platform events	Auto-subscribed
`user:<uuid>`	Notifications, security flags	Auto-subscribed, self-only
`conv:<uuid>`	Chat messages, typing indicators	Membership required
`party:<uuid>`	Playback sync, reactions	Membership required

Client -> Server Frames

```
{ "type": "ping" }
{ "type": "subscribe",   "room": "conv:<uuid>" }
{ "type": "unsubscribe", "room": "conv:<uuid>" }
{ "type": "typing",      "room": "conv:<uuid>" }
{ "type": "message.send", "room": "conv:<uuid>", "body": "hello" }
{ "type": "read",         "room": "conv:<uuid>" }
{ "type": "reaction",     "room": "party:<uuid>", "emoji": "?", "second_offset": 812 }
```

Server -> Client Events

Event	Room	Description
`hello`	--	Connection established
`presence`	global	User online/offline status

`platform.event`	global	Event bus envelope
`notification`	global + user	New notification
`security.flag`	user	Challenge/revoke decision
`typing`	conv/party	Typing indicator
`message.new`	conv	New message
`message.edited`	conv	Message was edited
`state.sync`	party	Playback sync state
`reaction`	party	Emoji reaction
`join.event` / `leave.event`	party	Member joined/left
`party.ended`	party	Party was ended
`pong`	--	Heartbeat response

PartySyncState

```
{
  "seq": 42,
  "playing": true,
  "position_seconds": 611.3,
  "server_time_ms": 1767225600123,
  "server_position_seconds": 623.9
}
```

Clients render from `server_position_seconds` and correct drift using `server_time_ms`.

Heartbeat

Server pings every 30s; connections silent for 75s are reaped.

Card Economy

Card Basics

Cards are cryptographic tokens (`SPC-{12hex}-{8hmac}`) that gate playback.

```
POST /api/cards/activate              # Activate a card
GET /api/me/card                      # Quick status check
GET /api/me/cards                     # Full card inventory
GET /api/me/cards/summary             # Summary with days remaining
```

Transfer

```
POST /api/cards/transfer-code         # Generate a 20-min one-time code
```

Merchant Operations

```
POST /api/merchant/register           # Register as a merchant
GET /api/merchant/inventory           # View inventory
POST /api/merchant/cards/purchase     # Purchase cards
POST /api/merchant/cards/give         # Give cards by username
POST /api/merchant/cards/redeem-transfer  # Redeem a transfer code
```

Admin Card Operations

```
POST /api/admin/cards/mint            # Mint cards in bulk
GET /api/admin/cards                  # Search/filter all cards
POST /api/admin/cards/grant           # Grant cards to a user
POST /api/admin/cards/{number}/hold   # Instant hold/release
GET /api/admin/cards/chain            # Full card chain visibility
GET /api/admin/billing/config         # View pricing config
PATCH /api/admin/billing/config       # Update pricing
```

```
GET /api/admin/card-purchases         # Purchase queue
POST /api/admin/card-purchases/{id}/approve  # Approve purchase
POST /api/admin/card-purchases/{id}/reject   # Reject purchase
GET /api/admin/merchants              # List merchants
POST /api/admin/merchants/{id}/lock   # Lock merchant
POST /api/admin/merchants/{id}/ban    # Ban merchant
```

Library & Playlists

Playlist CRUD

```
POST /api/library/playlists           # Create a playlist
GET /api/library/playlists            # List your playlists
GET /api/library/playlists/{id}       # Get playlist details
PATCH /api/library/playlists/{id}     # Update playlist
DELETE /api/library/playlists/{id}    # Delete playlist
```

Playlist Items

```
GET /api/library/playlists/{id}/items          # List items
POST /api/library/playlists/{id}/items          # Add item
PUT /api/library/playlists/{id}/items           # Replace all items
DELETE /api/library/playlists/{id}/items/{media_id}  # Remove item
PUT /api/library/playlists/{id}/order           # Reorder items
```

Storage

```
GET /api/library/storage              # Disk usage overview
```

File Management

File Listing

```
GET /api/files                        # List all files
GET /api/files/{id}                   # File details with probe data
```

Transcoding

```
POST /api/admin/files/{id}/transcode     # Enqueue transcode
GET /api/transcode-jobs/{id}              # Job status/progress
DELETE /api/transcode-jobs/{id}           # Cancel job
POST /api/admin/files/{id}/thumbnails     # Enqueue thumbnail extraction
```

HLS Streaming

```
GET /api/files/{id}/master.m3u8       # Dynamic master playlist
GET /api/hls/{job_id}/{*asset}        # HLS segment/playlist assets
```

Content Markers

```
GET /api/files/{id}/markers           # Read markers
PUT /api/files/{id}/markers           # Set markers (intro, outro, chapter)
POST /api/admin/files/{id}/markers/detect  # Auto-detect markers
GET /api/files/{id}/chapters          # Chapter list
```

Extension Resources

```
GET /api/extensions/{id}/catalog/{type}/{catalog_id}
GET /api/extensions/{id}/stream/{type}/{sid}
GET /api/extensions/{id}/meta/{type}/{mid}
GET /api/extensions/subtitles/{type}/{sid}
```

Ops & Observability

Health & Monitoring

```
GET /health                           # Liveness (plain "ok")
GET /api/ready                        # Deep readiness (DB/Redis/FFmpeg)
GET /api/stats                        # Dashboard: counts, sessions, transcodes
GET /metrics                          # Prometheus scrape endpoint
```

Automation Rules

```
GET /api/admin/automations            # List rules
POST /api/admin/automations           # Create rule
{ "event": "user.registered", "filter": {}, "action": "notify_all" }
PATCH /api/admin/automations/{id}     # Toggle enabled/disabled
DELETE /api/admin/automations/{id}    # Delete rule
```

Webhooks

```
GET /api/admin/webhooks               # List webhooks
POST /api/admin/webhooks              # Create webhook
DELETE /api/admin/webhooks/{id}       # Delete webhook
POST /api/admin/webhooks/{id}/test    # Test-fire webhook
```

Failure Analysis

```
GET /api/admin/failures               # Recent 5xx + per-route aggregates
```

Event Timeline

```
GET /api/admin/events?event=&limit=   # Platform events timeline
```

Notifications

Notifications are pushed to the global WS feed AND the recipient's personal `user:` room.

Billing Requests

```
POST /api/billing/requests            # Submit billing request
GET /api/billing/requests             # My requests
GET /api/admin/billing                # Admin queue
POST /api/admin/billing/{id}/approve  # Approve
POST /api/admin/billing/{id}/reject   # Reject
```

Developer Experience

API Versioning

Every response includes `X-API-Version: space/`. Legacy `/v1/*` paths auto-rewrite to `/api/*`.

OpenAPI

```
GET /api-docs/openapi.json            # Machine-readable spec
GET /docs                             # Swagger UI (interactive)
```

Error Format

All errors return `{ code, message, trace_id }` with correct HTTP status.

Pagination Convention

All list endpoints support `?limit=N&cursor=`. Response includes `next_cursor`.

Rate Limiting Headers

When rate-limited: `429` + `Retry-After: 60`.

Server Meta

```
GET /api/meta
# -> { version: "0.1.0", features: [...], docs: "/docs", websocket: "/ws" }
```

Deployment

- `docker-compose.prod.yml` -- Production stack (app + Postgres + Redis + nginx)

- `ops/nginx.conf` -- TLS-ready reverse proxy config
- `ops/backup.sh` -- Nightly backup with 14-day retention
- `ops/restore.sh` -- Restore with verification steps
- Graceful shutdown: SIGTERM drains in-flight requests and WS connections

WebSocket Protocol

Single endpoint: `GET /ws?token=` (session token; API keys are not accepted on the socket). On success the socket is upgraded; on failure you get an HTTP 401 before upgrade.

Framing

Every server->client message is JSON:

```
{ "type": "event", "room": "party:9f...", "data": { ... } }
```

Client->server frames are also JSON, one per text message.

Rooms

Room	Purpose
`__global__`	presence changes + platform event envelopes (auto-subscribed
`user:<your_uuid>`	your notifications, security flags, AI notices (auto-subscri
`conv:<conversation_id>`	direct/group/party chat messages, typing
`party:<party_id>`	playback sync, reactions, join/leave

Joining a room requires authorization -- the server validates every subscribe and every event: conversation/party membership tables for those rooms, strict self-match for `user:` rooms.

Client -> Server

```
{ "type": "ping" }
{ "type": "subscribe",   "room": "conv:..." }
{ "type": "unsubscribe", "room": "conv:..." }
{ "type": "typing",      "room": "conv:..." }
{ "type": "message.send", "room": "conv:<uuid>", "body": "hi" }
{ "type": "read",         "room": "conv:<uuid>" }
{ "type": "reaction",     "room": "party:<uuid>", "emoji": "?", "second_offset": 812 }
```

`message.send` persists to Postgres, fans out to all room subscribers as `message.new`, and replies with an ack:

```
{ "type": "message.sent", "data": { "id": "<message uuid>" } }
```

Errors come back inline:

```
{ "type": "error", "data": { "message": "not allowed to join this room" } }
```

Server -> Client event types

Event	Room	Payload (`data`)
`hello` (on connect)	global	`{ user_id, online_users[] }`
`presence`	global	`{ user_id, online, online_count }`
`platform.event`	global	`{ event, data }` -- universal event bus envelope (see below
`notification`	global + your `user:` room	Notification object
`security.flag`	your `user:` room	`{ decision: "challenge"| "revoke", score }`
`typing`	conv/party	`{ user_id, display_name }`
`message.new`	conv	full ChatMessage object
`reaction`	party	`{ user_id, emoji, second_offset }`
`state.sync`	party	PartySyncState (see below)
`join.event` / `leave.event`	party	`{ user_id, display_name? }`
`party.ended`	party	`{ party_id }`
`pong`	--	`{ ts }`

Platform event envelopes

The event bus (`events` table) bridges onto the global room:

```
{ "type": "event", "room": "__global__",
  "data": { "type": "platform.event",
            "event": "resolution.issued",
            "data": { "source": "...", "kind": "hls", "quality": "1080p" } } }
```

Current events: `resolution.issued`, `playback.authorized`, `progress.updated`.

New capabilities subscribe to these instead of coupling to producers; the timeline is also queryable at `GET /api/admin/events`.

PartySyncState

```
{ "seq": 42, "playing": true,
  "position_seconds": 611.3,        // position at last host action
  "server_time_ms": 1767225600123,  // wall clock when this was sent
  "server_position_seconds": 623.9  // projected NOW (position + elapsed)
}
```

Clients should render from `server_position_seconds` and correct drift by comparing their clock offset with `server_time_ms`.

Minimal JavaScript client

```
const ws = new WebSocket(`wss://host/ws?token=${token}`);
ws.onopen = () => ws.send(JSON.stringify({ type: "ping" }));
ws.onmessage = (e) => {
  const frame = JSON.parse(e.data);
  if (frame.type === "event") console.log(frame.room, frame.data.type, frame.data.data);
};
ws.send(JSON.stringify({ type: "subscribe", room: "conv:<uuid>" }));
ws.send(JSON.stringify({ type: "message.send", room: "conv:<uuid>", body: "hello" }));
```

Heartbeat (v2): the server sends a protocol-level ping every 30 s and reaps

connections that show no activity for 75 s -- clients should answer pongs (browsers do this automatically). Application-level `{"type":"ping"}` / `{"type":"pong"}` still works and also counts as liveness. Disconnects are clean -- rooms die with the socket and presence decrements automatically.

Media Graph

API.md · architecture.md

The media graph is a typed, directed relationship network connecting any media items -- regardless of type. An anime can link to its manga source, a novel to its film adaptation, a movie to its sequel, and so on.

Relationship Types

Relation	Meaning	Example
`adaptation_of`	This media is adapted from the other	Movie -> Book (the book came first)
`based_on`	Inspired by or loosely adapted from	Film -> True story
`sequel_of`	Direct continuation	Terminator 2 -> Terminator
`prequel_of`	Preceding story	Star Wars Ep I -> Ep IV
`spin_off_of`	Side story or derivative	Better Call Saul -> Breaking Bad
`remake_of`	New version of older work	2010 Karate Kid -> 1984 Karate Kid
`side_story_of`	Parallel narrative	Animated spinoff -> Main series
`parent_of`	Parent work that spawned children	Series -> Spinoff
`same_universe`	Shared fictional universe	Iron Man -> Captain America
`same_franchise`	Same franchise, different stories	Final Fantasy VII -> FFXVI
`alternate_version_of`	Different telling of same story	Manga -> Anime
`related_to`	Thematically or contextually related	Documentary -> Subject film

Directionality

Relationships are **directed**: `adaptation_of` means the *from* item was adapted *from* the *to* item. The graph API resolves both directions when traversing -- a query for "related" items returns both outgoing and incoming edges.

Cross-Media Relations

Five relations carry story across media formats (the "backbone" of cross-media journeys):

```
adaptation_of, based_on, spin_off_of, remake_of, alternate_version_of
```

These are used by the journey recommendation engine to suggest "the same story continues in another format."

Graph Operations

One-Hop Neighborhood

```
GET /api/media/{id}/related
```

Returns all directly connected media, grouped by relation direction (`out` = this media points at the other; `in` = the other points here). Ordered by `release_date`.

Franchise Traversal (BFS)

```
GET /api/media/{id}/journey
```

Breadth-first traversal up to 3 hops deep. Returns a flat list of `JourneyNode` objects, each with:

- `media_id`, `title`, `media_type`, `poster_url`, `release_date`
- `via_relation` -- the edge that led to this node (null for root)
- `depth` -- hops from root (0 = the root item)

The traversal:

1. Starts at the root media item
2. Explores all edges (both directions) at each depth level
3. Uses a `visited` set to prevent cycles
4. Limits to 4 hops maximum (clamped server-side)
5. Issues one SQL round-trip per depth level (bounded query count)

Cross-Media Journeys (User-Scoped)

```
GET /api/discover/journeys
```

Recommends story continuations the user hasn't consumed yet. Algorithm:

1. Find the user's **anchors**: recently completed or highly-rated items (score >= 7)
2. For each anchor, follow adaptation-style edges (`adaptation_of`, `based_on`, `spin_off_of`, `remake_of`, `alternate_version_of`) in both directions
3. Filter out items the user has already watched or read
4. Return up to `limit` journeys with the "reason" string: *"you finished {title} -- the same story continues in another format"*

CRUD

```
# Create a relationship (admin)
POST /api/media/{id}/relationships
{ "to_media_id": "...", "relation": "adaptation_of", "source": "tmdb" }

# List all relationships for a media item
GET /api/media/{id}/relationships
# -> { "outgoing": [...], "incoming": [...] }

# Delete a relationship (admin)
DELETE /api/media/{id}/relationships/{to_media_id}/{relation}
```

Database Schema

```
CREATE TABLE media_relationships (
    id          UUID PRIMARY KEY,
    from_media_id UUID NOT NULL REFERENCES media(id),
    to_media_id   UUID NOT NULL REFERENCES media(id),
    relation    TEXT NOT NULL,  -- CHECK: one of the 12 relation types
    source      TEXT NOT NULL DEFAULT 'manual',
    confidence  DOUBLE PRECISION NOT NULL DEFAULT 1.0,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (from_media_id, to_media_id, relation)
);
```

- `source` tracks who created the relationship: `manual`, `tmdb`, `ai_copilot`, `plugin`, etc.
- `confidence` defaults to 1.0 (manual); AI-sourced links may have lower confidence
- Upsert semantics: re-creating the same `(from, to, relation)` updates the `source`

Population

Relationships are populated from:

1. **Manual admin entry** -- via the API
2. **TMDB metadata import** -- related titles, franchises, collections
3. **AI Copilot** -- automatic graph enrichment (whitelisted action)
4. **Plugin imports** -- some metadata plugins supply relationship data

Metrics

Graph operations don't emit dedicated metrics. Query performance is tracked via the general `http_request_duration_seconds` histogram.

Limitations

- Maximum traversal depth: 4 hops (hardcoded clamp)
- No weighted edges (confidence exists but isn't used in traversal ranking)
- Cycle-safe via visited set (no infinite loops)
- No graph-wide analytics (pagerank, clustering) -- this is a homelab server, not a social network

Playback Security Model

API.md · SECURITY.md · SECURITY-AUDIT.md

This document details the multi-layered playback protection system. Every layer is optional and off by default -- enable via admin policy.

Architecture Overview

```
Client request
    |
    ?
+------------------+
|  1. Card Gate     |  No card? -> 403 (everyone, including admins)
|  ensure_card_access
+----------?-------+
           |
           ?
+------------------+
|  2. Auth Check    |  Bearer token / API key / session
+----------?-------+
           |
           ?
+------------------+
|  3. Risk Engine   |  Signal scoring -> Allow / Challenge / Revoke
|  (opt-in)        |  "off" until admin enables risk_policies
+----------?-------+
           |
           ?
+------------------+
|  4. Grant Mint    |  HMAC-signed, short-lived (60-3600s)
|  PlaybackGrant   |  Bound to user + media + quality + device
+----------?-------+
           |
           ?
+------------------+
|  5. Stream        |  Watermarked URLs, segment-scoped grants,
|  Protection      |  offline licenses, key injection
+------------------+
```

Layer 1: Card Gate

Nothing plays without an active, unexpired access card. This includes admins.

```
// src/cards.rs:ensure_card_access
SELECT EXISTS (
  SELECT 1 FROM access_cards
  WHERE owner = $1 AND status = 'active' AND expires_at > now()
)
```

- Cards are `SPC-{12hex}-{8hmac}` -- the last 8 chars are an HMAC-SHA256 checksum

- Trial cards (7 days) are issued once per user on registration
- Default card duration: 30 days
- Brute-force protection: 5 failures in 15 minutes -> 15-minute lockout

Layer 2: Authentication

Three methods, all accepted:

Method	Header	Use Case
Bearer token	`Authorization: Bearer <jwt>`	Primary (SPA, mobile)
API key	`X-API-Key: sk_...`	Machine-to-machine
Session cookie	`Cookie: space_session=...`	Browser

Tokens are opaque (not JWTs), SHA-256 hashed at rest, with 15-min access TTL and 30-day rotating refresh TTL.

Layer 3: Risk Engine

Disabled by default. When off, any authenticated user with a valid card gets a grant.

When enabled, the engine scores signals and maps to decisions:

Signals

Signal	Scoring	Max Points
`concurrent_sessions`	25 base + 12 per session over limit (default: 2)	50
`distinct_devices_24h`	15 base + 8 per device over limit (default: 3)	40
`resolves_per_hour`	20 base + 4 per resolve over limit (default: 5)	45
`prior_flags_24h`	10 × count of non-allow security events	30
`distinct_client_contexts_24h`	18 base + 7 per context over limit (default: 3)	35

Decision Bands

Score Range	Decision	Effect
0 -> threshold	`Allow`	Grant issued normally
threshold -> revoke_threshold	`Challenge`	Client must acknowledge; new fingerprint recorded
> revoke_threshold	`Revoke`	All grants for user revoked; session killed

Default thresholds: `challenge=40`, `revoke=80` (configurable via `PATCH /api/admin/security/policy`).

Device Fingerprinting

Fingerprint priority (checked in order):

1. `X-JA4` header (edge-computed TLS fingerprint)
2. `X-Client-Fingerprint` header (client-computed)
3. `fingerprint` field in request body

Fingerprints are SHA-256 hashed before storage (never raw). Used for:

- Device counting (distinct devices in 24h)
- Grant binding (segment grants check device hash)

- Offline license binding

Layer 4: Grant System

Grant Lifecycle

```
POST /api/playback/session
    |
    +- verify card access
    +- authenticate user
    +- risk assessment (if enabled)
    +- issue HMAC-signed grant
    +- return { grant, expires_in }
         |
         ?
    Client uses grant for playback
         |
         +- GET /api/files/{id}/raw?grant=...
         +- GET /api/playback/keys/{file_id}?grant=...
         +- POST /api/playback/grant/segment (for sub-grants)
```

Grant Token Format

The grant token is an HMAC-SHA256 signature over a pipe-delimited payload:

```
grant_id|user_id|media_ref|quality|exp|scope
```

- `scope` is empty for full-access grants, or `seg:` for segment-scoped grants
- Verified using constant-time comparison (`subtle::ConstantTimeEq`)
- Server-side state: every full-access grant has a DB row for revocation checking

Grant Parameters

Parameter	Default	Range	Description
`ttl`	600s	60-3600s	Grant lifetime
`quality`	--	`sd`, `hd`, `fhd`, `uhd`	Maximum allowed quality
`device_hash`	--	SHA-256	Bound to specific device

Segment-Scoped Grants

For HLS streaming, the parent grant can mint child grants scoped to specific segments:

```
POST /api/playback/grant/segment
{ "authorization": "<parent-grant>", "segment": 42 }
```

- Child grant TTL: 120 seconds
- Scope: `seg:42`
- The parent grant must still be valid
- Prevents segment prefetching beyond the current playback position

Grant Revocation

```
# Revoke a specific grant (admin)
POST /api/admin/security/grants/{id}/revoke

# Revoke all grants for a user (admin)
POST /api/admin/security/users/{user_id}/revoke-grants
```

Revocation is immediate -- the grant ID is checked on every use.

Layer 5: Stream Protection

Forensic Watermarking

Every resolved stream URL carries a `wm=` parameter encoding:

- User ID
- Session reference
- Hourly time bucket

```
// src/security/watermark.rs
HMAC-SHA256(secret, user_id || session_ref || hourly_bucket)[..16]
```

If a stream is pirated, the watermark identifies the source user and time window.

HLS Key Injection

For HLS streams with encryption:

```
GET /api/playback/keys/{file_id}?grant=<grant_token>
```

Returns the AES-128 content key for decrypting HLS segments. The key is derived from the media key via HKDF-SHA256.

Key Hierarchy

```
Master Secret (MASTER_KEY env or DB-generated)
    |
    +- derive_kek(master, "media-key-wrap")
    |       |
    |       +- AES-256-GCM wrap/unwrap media keys
    |
    +- derive_kek(master, "hls-segment-aes128")
            |
            +- HKDF -> 16-byte HLS content key
```

- Master secret: from `MASTER_KEY` env var, or auto-generated and stored in `signing_secrets` table
- Media keys: 32-byte random keys, one per media item, versioned for rotation
- Content keys: 16-byte AES-128 keys derived from media keys

Key Rotation

```
POST /api/admin/security/keys/{media_ref}/rotate
```

Creates a new version of the media key, retiring the old one. Old segments encrypted with the old key remain playable until their grants expire.

Key Revocation (Instant)

```
POST /api/admin/security/keys/{media_ref}/revoke
```

Immediately revokes all keys for a media item. All active grants for that media become useless.

Offline Licenses

For downloads (plane mode, etc.):

```
POST /api/playback/offline-keys
{ "media_ref": "...", "device_id": "...", "fingerprint": "..." }
```

- Returns the content key directly (the client stores it)
- TTL: 1-7 days (default 24h)
- Bound to user + media + device (all three must match on validation)
- Idempotent: re-requesting within the validity window returns the same license
- Revocable: `DELETE /api/playback/offline-keys/{id}`

Lockdown Mode

Emergency site-wide lockdown:

```
POST /api/admin/security/lockdown
{ "enabled": true, "message": "Maintenance in progress" }
```

- All public routes return 503 with the custom message
- Cached in-process with 5-second TTL (fast toggle)
- Stored in `site_lockdown` table (survives restarts)
- Admin access unaffected

Security Events

Every risk decision is logged to `security_events`:

```
CREATE TABLE security_events (
    id          UUID PRIMARY KEY,
    user_id     UUID REFERENCES users(id),
    signal_type TEXT,
    severity    TEXT,
    score       DOUBLE PRECISION,
    decision    TEXT,
    details     JSONB,
    created_at  TIMESTAMPTZ DEFAULT now()
);
```

Admin audit: `GET /api/admin/security/events?action=&limit=`

Honeypot

`GET /api/admin/debug/dump-secrets` is a trap. Any request triggers:

1. IP hash banned in `admin_bans`
2. Authenticated user's sessions wiped
3. Watchdog trip

Configuration

Variable	Default	Effect
`ADMIN_LOCAL_ONLY`	`true`	Restrict admin routes to loopback
`ACCESS_TOKEN_TTL_SECS`	`900`	Access token lifetime
`REFRESH_TOKEN_TTL_SECS`	`2592000`	Refresh token lifetime (30 days)
`PASSWORD_LOCKOUT_ATTEMPTS`	`5`	Failed login lockout threshold
`PASSWORD_LOCKOUT_MINS`	`15`	Lockout duration

Risk policy is tuned at runtime via `PATCH /api/admin/security/policy` -- no restart needed.

Mood Search & Smart Collections

API.md · heuristic-engine.md · architecture.md

SPACE's intelligence module provides natural-language mood matching ("vibe search"), taste profiling, home page rails, and user-created smart collections.

Vibe Search

Search for media by mood, tone, or feeling rather than keywords.

```
GET /api/intelligence/vibe?q=cozy+feel-good&limit=12
```

How It Works

1. **Query normalization** -- lowercase, split on whitespace
2. **Vibe matching** -- matches query words against a taxonomy of 12 moods (see below)
3. **Genre union** -- collects all genres from matched vibes, deduplicated
4. **Candidate query** -- fetches media matching any of those genres, ordered by popularity
5. **Scoring** -- each candidate is scored by:

- Base: `rating_avg + popularity/2000`
- +1.5 per keyword match in title or overview
- +0.75 if runtime falls within the vibe's pace band

6. **Ranking** -- sort by score descending, return top N

The 12 Vibes

Vibe	Label	Genres	Keywords	Pace (min)
`Cozy`	Cozy & feel-good	comedy, family, romance,	warm, charming, delightful,	80-130
`Dark`	Dark & gritty	crime, thriller, drama, mystery	dark, brutal, corrupt, noir,	95-160
`MindBending`	Mind-bending	science-fiction, sci-fi, fantasy,	time travel, parallel, dream,	100-170
`Epic`	Epic adventure	adventure, action, fantasy,	epic, journey, kingdom, war,	110-190
`Tearjerker`	Emotional rollercoaster	drama, romance	loss, grief, love story, illness,	90-150
`Funny`	Laugh out loud	comedy	hilarious, satire, misfit, wedding	75-120
`Scary`	Scary as hell	horror, thriller	haunted, demon, supernatural,	80-130
`Chill`	Chill & easy	documentary, music, comedy,	nature, food, travel, road trip	60-110
`Nostalgic`	Nostalgia trip	family, animation, adventure,	childhood, summer, coming of	85-140
`Intense`	Edge of your seat	thriller, action, crime	race against, hostage, heist,	90-150
`Romantic`	Romance fix	romance, drama, comedy	fall in love, wedding, first kiss,	85-135
`Cerebral`	Slow-burn cerebral	drama, mystery, science-fiction	philosophical, identity, existential, quiet, isolation	100-180

Response Format

```
{
  "matched_vibes": ["Cozy", "Funny"],
  "items": [
    {
```

```
      "media_id": "...",
      "title": "Paddington 2",
      "poster_url": "...",
      "score": 8.7,
      "why": "Cozy & feel-good · charming · wholesome · under 130 min"
    }
  ]
}
```

Listing Available Moods

```
GET /api/intelligence/moods
# -> [{ "id": "cozy", "label": "Cozy & feel-good" }, ...]
```

Taste Profile

```
GET /api/me/taste
```

Returns a profile built from the user's watch history and list entries:

```
{
  "top_genres": [
    { "genre": "science-fiction", "weight": 85.2 },
    { "genre": "drama", "weight": 62.1 }
  ],
  "favorite_decades": [2020, 2010, 1990],
  "avg_runtime_minutes": 128,
  "anime_affinity": 32.5,
  "total_watch_hours": 847,
  "completion_rate": 72.4
}
```

How it's built:

1. Genre weights from watch history (weighted by count) + list entries (weighted by score, minimum 5.0)
2. Merged, normalized to 0-100, truncated to top 6
3. Favorite decades: top 3 by entry count
4. Anime affinity: percentage of anime genre entries
5. Completion rate: completed / (completed + watching + dropped)

Home Rails

```
GET /api/home
```

Returns 7 curated rails for the home screen:

Rail	Source	Count
`continue_watching`	Watch history with resume positions	Up to 12
`trending`	Windowed popularity (week)	Up to 12
`because_you_watched`	Similar items to highest-scored recent watch	Up to 8
`new_episodes`	Episodes airing in next 14 days for user's shows	Up to 8
`friends_activity`	Activity from followed users	Up to 8
`recommended`	Recommendation engine output	Up to 12

`vibe_picks`	Hardcoded `vibe_search("cozy feel-good", 8)`	8

The `because_you_watched` rail picks the user's highest-scored watching/completed entry as the anchor, then fetches similar items.

Smart Collections

User-created, rule-based media collections that re-evaluate dynamically.

Creating a Collection

```
POST /api/collections
{
  "name": "Hidden Gems",
  "description": "Underrated sci-fi from the 90s",
  "rules": {
    "media_type": "movie",
    "genres": ["science-fiction"],
    "year_from": 1990,
    "year_to": 1999,
    "min_rating": 7.0,
    "is_anime": false,
    "sort": "rating"
  },
  "visibility": "public"
}
```

Rule Schema

Field	Type	Description
`media_type`	`string`	`movie`, `series`, `anime`, `book`, etc.
`genres`	`string[]`	Up to 12 genre names
`year_from`	`int`	Minimum release year
`year_to`	`int`	Maximum release year
`min_rating`	`float`	Minimum average rating
`is_anime`	`bool`	Filter to anime only
`sort`	`string`	`popularity` (default), `rating`, `title`, `created_at`

Viewing Collection Items

```
GET /api/collections/{id}/items
```

The rules are evaluated against the `media` table at query time -- the collection always reflects the current catalog. Visibility is enforced: `public` collections are viewable by anyone, `private` only by the owner.

Management

```
# List your collections
GET /api/me/lists  (or use the intelligence endpoints)

# Delete a collection
DELETE /api/collections/{id}
```

Constraints:

- Name: 1-48 characters
- Max 12 genres per rule
- Visibility: `private` (default) or `public`
- Only the owner can delete

AI Copilot

The AI copilot can automatically enrich the catalog with relationship data:

```
# Check status
GET /api/admin/ai/status

# Trigger analysis
POST /api/admin/ai/analyze

# Apply suggested changes
POST /api/admin/ai/apply

# View audit log
GET /api/admin/ai/events
```

The copilot is configured at runtime -- no restart needed:

```
GET /api/admin/ai/config
PATCH /api/admin/ai/config
```

Smart Collections vs. Manual Lists

Feature	Smart Collections	Manual Lists
Content	Rule-based, auto-populated	Manually curated
Updates	Dynamic (re-evaluated on view)	Static (add/remove items)
Sharing	Public or private	Shareable via list ID

| Use case | "Show me all 90s sci-fi rated 7+" | "My favorite movies"

Heuristic Engine

SPACE's built-in learning system -- the brain of a Spotify/Netflix-class platform, running natively on your own data. No external AI service, no connectors to other companies' apps: the engine silently learns from every in-app action (progress saves, completions, rewatches, ratings, favorites) and turns them into a living behavioral profile, Wrapped-style recaps, deterministic insights and explainable next-up picks.

What it learns (and how)

Signal	Technique
Genre / decade affinity	engagement weights with **exponential time-decay** (21-day h
Top titles	decayed play-frequency with replay counts
Habits	circular-smoothed hour-of-day + weekday histograms -> **chro
Sessions	gap-based reconstruction (45 min threshold) with binge detec
Replay behavior	replay rate -> comfort-replayer vs **explorer index**
Completion	percent-played discipline across plays vs completions
Streaks & momentum	current/longest active-day streaks, 7-day week-over-week del
Personas	multi-label rule classifier with confidences -- Cinephile, S

Signal weights: `favorite` 3.0 · `complete` 2.0 · `rewatch` 1.5 · `rate` 1.25 · `play` 1.0.

Automatic feeding

The engine hooks the existing write paths -- nothing for clients to call:

- `PUT /api/progress` -> play/complete signals (saves inside one hour dedupe to one event)
- `PUT /api/me/media/{id}/status` as completed -> completion signal
- `PUT /api/media/{id}/rating` -> rating signal
- `POST /api/media/{id}/favorite` -> favorite signal

API

```
GET    /api/heuristic/status              engine liveness + learned-signal count
GET    /api/me/heuristic                  full learned profile
GET    /api/me/heuristic/wrapped?period=  Wrapped recap (week|month|year|all)
GET    /api/me/heuristic/insights         human-readable deterministic insights
GET    /api/me/heuristic/predictions      explainable next-up picks + reasons
DELETE /api/me/heuristic                  wipe all my learned data
```

Wrapped response highlights

`headline`, total plays/hours, unique titles, new discoveries (first-ever engagements in the window), top items/genres/decades, biggest single day, longest session, active days.

Prediction scoring

`(0.65 × genre affinity + 0.35 × popularity) × freshness`, excluding finished titles and just-watched items; every pick ships reasons like *"matches your taste for Thriller & Mystery"*, *"new to you"*, *"back after a while"*.

Example

```
curl http://localhost:8080/api/me/heuristic -H "Authorization: Bearer $TOKEN"
# { "personas": [{"label":"Music Devotee","confidence":0.9}, ...],
#   "habits": {"chronotype":"night owl","peak_hours":[22,23,20], ...},
#   "genre_affinity": [{"key":"Sci-Fi","weight":100.0,"plays":84}, ...] }

curl "http://localhost:8080/api/me/heuristic/wrapped?period=year" -H "Authorization: Bearer $TOKEN"
# { "headline": "This Year was a deep-dive into 2010s Sci-Fi: 212.4 hours ...",
#   "top_items": [...], "new_discoveries": 61, "biggest_day": {...} }
```

Card Economy

API.md · SECURITY.md

SPACE uses an access-card system to gate playback. Every user -- including admins -- must have an active card to stream.

Card Basics

A card is a cryptographic token in the format:

```
SPC-{12hex}-{8hmac}
```

The first group is a random body. The second group is an HMAC-SHA256 checksum over the body, truncated to 8 hex characters. This means:

- **Tamper detection:** Any modification to the card number is caught offline
- **Forgery prevention:** Creating valid checksums requires the server-side signing secret
- **Format verification:** `verify_card_format()` recomputes the HMAC and does constant-time comparison

Card Lifecycle

```
+----------+     +----------+     +----------+
|  Minted  |----?|  Active  |----?| Expired  |
|  (issued) |     | (playing) |     |          |
+----------+     +-----?----+     +----------+
                       |
                       +- admin hold --? On Hold
                       |
                       +- admin revoke --? Revoked
```

Status	Meaning
`active`	Normal, usable for playback
`expired`	Past `expires_at` timestamp
`held`	Admin has frozen the card (temporary)
`revoked`	Permanently disabled

Trial Cards

Every new user automatically receives a 7-day trial card on registration:

```
// Issue exactly once per user (partial unique index enforces this)
INSERT INTO access_cards (owner, issued_by, duration_days)
VALUES ($1, 'trial', 7)
-- ON CONFLICT (owner) WHERE issued_by = 'trial' -> no-op
```

Trial cards are issued at registration time. If a user somehow loses their trial (e.g., database manipulation), the partial unique index prevents re-issuance.

Minting Cards

Admin Minting

```
POST /api/admin/cards/mint
{ "count": 100, "duration_days": 30 }
```

Mints up to 500 cards at once. Each card gets:

- A unique `SPC-{body}-{hmac}` number
- Full HMAC stored separately (`number_hmac`) for indexed lookups
- `issued_by: "admin"`
- `duration_days` (clamped 1-365)

Merchant Minting

Merchants purchase cards with cryptocurrency, then admin approves:

```
# Merchant registers
POST /api/merchant/register

# Merchant purchases cards
POST /api/merchant/cards/purchase
{ "count": 10, "payment_ref": "btc-tx-hash" }

# Admin approves the purchase
POST /api/admin/card-purchases/{id}/approve
```

Activation

Activating a Card

```
POST /api/cards/activate
{ "card_number": "SPC-A1B2C3D4E5F6-D7E8F9A0" }
```

Activation hardening:

- 5 failed attempts in 15 minutes -> 15-minute lockout
- Lockout is persisted (survives restarts)
- Failed attempts are tracked per (user, IP) pair
- Metric: `card_activations_total{result="success|failure"}`

On activation:

1. Card number is verified (HMAC check)
2. Card must be `active` status and unexpired
3. Card must not already have an owner (or must be the same owner)
4. `expires_at` is set to `now() + duration_days`
5. Card is bound to the activating user

Transfer

Users can gift cards to each other via one-time transfer codes:

```
# Generate a transfer code (valid 20 minutes)
POST /api/cards/transfer-code
```

```
# -> { "code": "...", "expires_at": "..." }

# Redeem on the recipient's account
POST /api/merchant/cards/redeem-transfer
{ "code": "..." }
```

Transfer codes are single-use and time-limited.

Merchant Workflow

Registration

```
POST /api/merchant/register
```

Creates a merchant record. Merchants can:

- View their inventory
- Purchase cards (pending admin approval)
- Give cards to users by username
- Redeem user transfer codes

Inventory

```
GET /api/merchant/inventory
# -> { "available": 42, "given": 15, "pending": 3 }
```

Giving Cards

```
POST /api/merchant/cards/give
{ "username": "alice", "count": 5 }
```

Approval Flow

1. Merchant submits purchase request with crypto payment reference
2. Request enters pending queue
3. Admin reviews: `POST /api/admin/card-purchases/{id}/approve` or `reject`
4. On approval: cards are minted and added to merchant inventory

Admin Operations

Card Search

```
GET /api/admin/cards?owner=&status=&limit=50
```

Grant Cards Directly

```
POST /api/admin/cards/grant
{ "user_id": "...", "count": 5, "duration_days": 30 }
```

Instant Hold/Release

```
POST /api/admin/cards/{card_number}/hold
{ "hold": true }
```

Hold is temporary -- the card resumes when released.

Card Chain Visibility

```
GET /api/admin/cards/chain?card_number=...
# -> Full lineage: who minted it, who gave it, who holds it
```

Merchant Management

```
# List merchants
GET /api/admin/merchants

# Lock a merchant (freeze inventory)
POST /api/admin/merchants/{id}/lock

# Ban a merchant (permanent)
POST /api/admin/merchants/{id}/ban
{ "reason": "fraud" }
```

Billing Configuration

```
# View pricing
GET /api/admin/billing/config
# -> { "card_price_usd": 9.99, "trial_days": 7, ... }

# Update pricing
PATCH /api/admin/billing/config
{ "card_price_usd": 14.99 }
```

Quick Status Check

```
GET /api/me/card
# -> { "active": true, "expires_at": "...", "days_remaining": 23 }

GET /api/me/cards/summary
# -> { "total": 3, "active": 2, "expired": 1, "days_remaining": 23 }
```

Key Metrics

Metric	Labels	Description
`playback_blocked_no_card_total`	--	Playbacks rejected (no card)
`card_activations_total`	`result`	Activation attempts
`cards_minted_total`	`issued_by`	Cards created (`admin`, `trial`, `merchant`)

Security Properties

- Card numbers are not sequential or predictable (random hex body)
- HMAC prevents offline forgery
- Constant-time comparison prevents timing attacks
- Brute-force protection with lockout
- Full chain of custody visible to admins
- Admin can hold/revoke any card instantly
- Cards expire server-side (no client-side expiration)

Automation & Webhooks

API.md · architecture.md

SPACE includes an in-process automation engine and outbound webhook system for reacting to platform events.

Automation Rules

Automation rules follow a **WHEN / IF / THEN** pattern:

```
WHEN event fires
  IF payload matches filter
    THEN execute actions
```

Creating a Rule

```
curl -X POST /api/admin/automations \
  -H "Authorization: Bearer $ADMIN" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Notify on new movies",
    "when_event": "media.added",
    "filter": { "media_type": "movie" },
    "actions": [
      { "type": "notify_all", "title": "New Movie", "body": "A new movie was added" }
    ]
  }'
```

Supported Events

Event	Fires When	Payload Fields
`media.added`	New media imported into catalog	`media_id`, `title`, `media_type`
`playback.started`	User begins playback	`user_id`, `media_id`, `source`
`payment.approved`	Card purchase approved	`user_id`, `card_count`, `merchant_id`

Filter Syntax

Filters are JSON objects with equality matchers. Every key in the filter must appear in the event payload with an equal value:

```
{ "media_type": "movie" }
{ "media_type": "movie", "source": "ia-films" }
```

An empty object `{}` matches all events (no filtering).

Action Types

Type	Parameters	Description

`notify_all`	`title`, `body`	Push notification to all active users
`notify_users`	`user_ids[]`, `title`, `body`	Push notification to specific users
`webhook`	`event`, `payload`	Trigger an outbound webhook

Example with multiple actions:

```
{
  "name": "Movie added -> notify + webhook",
  "when_event": "media.added",
  "filter": { "media_type": "movie" },
  "actions": [
    { "type": "notify_all", "title": "New Movie!", "body": "Check out the new addition" },
    { "type": "webhook", "event": "automation.custom.movie_alert", "payload": {} }
  ]
}
```

Managing Rules

```
# List all rules
GET /api/admin/automations

# Toggle a rule on/off
PATCH /api/admin/automations/{id}
{ "active": true }

# Delete a rule
DELETE /api/admin/automations/{id}
```

Execution Model

- Rules are evaluated in-process (no external job queue)
- `dispatch()` is called from the same trigger points that fire webhooks
- **Failures never propagate** -- if a rule errors, it's logged and skipped
- Each execution increments the rule's `run_count` and emits `automation_runs_total`
- Rules are ordered by `created_at` and evaluated sequentially

Outbound Webhooks

Webhooks deliver HMAC-SHA256 signed JSON payloads to external endpoints.

Creating a Webhook

```
curl -X POST /api/admin/webhooks \
  -H "Authorization: Bearer $ADMIN" \
  -H "Content-Type: application/json" \
  -d '{ "url": "https://hooks.example.com/space", "events": "*" }'
```

Response:

```
{
  "id": "...",
  "secret": "a1b2c3...",
  "note": "shown once; sign header is X-Signature (hmac-sha256 of raw body)"
}
```

> ***The secret is shown only once at creation.** Store it securely.*

URL Requirements

- Must be `https://` (production)
- `http://localhost` and `http://127.0.0.1` allowed for testing

Event Filtering

`events` Value	Meaning
`*`	Receive all events
`media.added`	Only media import events
`playback.started`	Only playback events
`payment.approved`	Only payment events

Payload Format

Every webhook receives:

```
{
  "event": "media.added",
  "ts": "2025-01-15T12:00:00Z",
  "data": { ... }
}
```

Signature Verification

Each request includes:

Header	Value
`X-Signature`	HMAC-SHA256 hex digest of the raw JSON body, keyed with the
`X-Event`	The event name
`Content-Type`	`application/json`

To verify:

```
import hmac, hashlib

def verify_webhook(body: bytes, signature: str, secret: str) -> bool:
    expected = hmac.new(secret.encode(), body, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, signature)
```

Delivery Behavior

- **Timeout:** 8 seconds per attempt
- **Retries:** 3 attempts with linear backoff (800ms, 1600ms, 2400ms)
- **Fire-and-forget:** Delivery runs in a spawned Tokio task; caller doesn't wait
- **Status tracking:** `last_status` (HTTP code or `network_error`) and `last_fired_at` are stored on the webhook record

Metrics

```
webhook_deliveries_total{event="media.added", status="200"} 12
```

```
webhook_deliveries_total{event="media.added", status="500"}  2
```

Managing Webhooks

```
# List (secrets masked)
GET /api/admin/webhooks

# Test-fire (sends { "test": true } with event "*")
POST /api/admin/webhooks/{id}/test

# Delete
DELETE /api/admin/webhooks/{id}
```

Automation -> Webhook Bridge

The `webhook` action type in automation rules bridges the two systems. When an automation rule fires a `webhook` action, it calls `webhooks::emit()` with the specified event name, which then delivers to all matching webhook subscriptions.

This means you can create compound workflows:

1. `media.added` fires automation rule
2. Rule matches filter `{ "media_type": "movie" }`
3. Rule's `webhook` action fires with event `automation.custom.new_movie`
4. External webhook endpoint receives the signed delivery

Observability

Both systems emit Prometheus metrics:

Metric	Labels	Description
`automation_runs_total`	`event`	Rules executed per event type
`webhook_deliveries_total`	`event`, `status`	Delivery attempts and results

View recent activity:

```
GET /api/admin/events?event=automation.executed&limit=20
```

Plugin Guide

SPACE is extensible at runtime through **HTTP plugins**: small services that speak a JSON protocol. Write them in any language (Node, Python, Go, Rust...), host them anywhere, register them from the admin console (`/admin.html`) or the API -- no server restart.

Two kinds:

kind	Purpose	Protocol
`metadata`	Search + detail lookups (like built-in	`/search`, `/details`
`source`	Import an entire external library into the catalog	`/items`

Registering

```
curl -X POST localhost:8080/api/admin/plugins \
  -H "Authorization: Bearer $ADMIN" -H 'Content-Type: application/json' \
  -d '{ "name":"my-anime-db", "kind":"metadata",
        "base_url":"https://plugins.example.com/mydb", "enabled":true }'
```

Rules: `name` = unique slug `[a-zA-Z0-9_-]{1,48}` and becomes the source key; `base_url` must be absolute http(s), no trailing slash. Registered plugins are hot-loaded -- they appear instantly in `GET /api/metadata/sources`.

Metadata plugin protocol

`GET {base}/search?q=`

```
{ "results": [
  { "id": "42",
    "title": "Cowboy Bebop",
    "overview": "...",
    "year": 1998,
    "poster": "https://.../poster.jpg",
    "type": "series" }
]}
```

- `id`: your plugin's stable external id (string)
- `type`: `movie` | `tv` | `anime` (hint only)

`GET {base}/details?id=`

```
{
  "title": "Cowboy Bebop",
  "original_title": "?????????",
  "overview": "...",
  "release_date": "1998-04-03",
  "runtime_minutes": 24,
```

```
  "status": "finished",
  "media_type": "anime",
  "poster": "https://...", "backdrop": "https://...",
  "genres": ["action", "sci-fi", "western"],
  "studios": ["Sunrise"],
  "cast": [ { "name": "Steve Blum", "character": "Spike" } ]
}
```

Only `title` is mandatory; everything else degrades gracefully.

`media_type:"anime"` sets the anime flag on import. Statuses map to:

`announced releasing released ongoing finished cancelled`.

What import does

`POST /api/metadata/import {"source":"my-anime-db","external_id":"42"}` will create or update the catalog item, attach genres/studios/cast, store the mapping `(source=my-anime-db, external_id=42)` so re-imports update in place, and return the full MediaDetail.

Source plugin protocol

A source exposes its whole library; sync pulls it into the catalog:

`GET {base}/items`

```
{ "items": [
  { "id": "book-123", "title": "Dune", "type": "book",
    "year": 1965, "overview": "...", "poster": "https://..." },
  { "id": "mov-9",    "title": "Arrival", "type": "movie", ... }
]}
```

Then trigger (or schedule) the sync:

```
curl -X POST localhost:8080/api/admin/plugins/<PLUGIN_ID>/sync \
     -H "Authorization: Bearer $ADMIN"
-> { "imported": 128, "skipped": 12 }
```

Skips items already mapped (dedupe by `(plugin name, id)`). Re-running is idempotent; new items get added.

Minimal Node.js metadata plugin (~30 lines)

```
import express from "express";
const app = express();
const db = [ // your data layer here
  { id: "1", title: "Night Raid 1931", type: "anime", year: 2010,
    overview": "...", genres: ["action","thriller"] }
];

app.get("/search", (req, res) => {
  const q = String(req.query.q ?? "").toLowerCase();
  res.json({ results: db.filter(x => x.title.toLowerCase().includes(q)) });
```

```
});

app.get("/details", (req, res) => {
  const hit = db.find(x => x.id === req.query.id);
  if (!hit) return res.status(404).json({ error: "not found" });
  res.json({ ...hit, release_date: `${hit.year}-01-01` });
});

app.listen(7001);
console.log("plugin on :7001");
```

Register with `{"name":"night-raid","kind":"metadata","base_url":"http://127.0.0.1:7001"}`.

Lifecycle & ops notes

- Timeouts: 10 s per request (metadata), 30 s for source sync pages
- Redirects are not followed; respond directly
- Plugin failures are isolated: errors surface as `503 unavailable` on the specific call and never take down core routes
- Disable without deleting via `PATCH ... {"enabled":false}`
- `POST .../{id}/test` runs a live search probe and reports counts

Python Source Plugins

SPACE treats every source the same way -- native compiled-in providers and remote HTTP plugins. That means **any language** can be a source, including Python. We ship a ready-to-run template: `plugins/python/space_plugin.py`.

Quickstart

```
python3 plugins/python/space_plugin.py --port 8901
```

Then register it (admin console -> *Plugins & Sources* -> paste URL):

```
http://localhost:8901
```

SPACE probes `manifest.json` -> `/items` -> `/search` and registers whichever protocol it speaks. No restarts, no compilation.

The contract

Route	Purpose	Response
`GET /manifest.json`	identity	`{ name, label, category, transport }`
`GET /items?limit=`	browse feed	`{ items: [SourceItem] }`
`GET /search?q=&limit=`	search	`{ results: [SourceItem] }`
`GET /details?id=`	detail + streams	`{ title, overview?, year?, poster?, media_type, streams: [{`

`SourceItem`: `{ id, title, overview?, year?, poster?, type }`
(`type`: movie · series · book · music · manga)

Stream kinds SPACE understands: `mp4`, `hls`, `audio`, `page`, `file`.

How it runs server-side

Once registered, your plugin participates in everything automatically:

- unified search fan-out (`/api/search/all`) with per-source timeouts,
- circuit breaking (3 consecutive failures -> skipped until prober recovers),
- telemetry (`/api/sources/status` shows its latency + success rates),
- resolver preference ranking and title-based failover.

Security notes

- HTTPS URLs for `base_url` are strongly recommended; auth via the optional `auth_token` confi

g field is sent as `Authorization: Bearer ...`.

- Plugins are trusted code hosts: run them in their own container/user.
- The plugin never receives user credentials -- only anonymous queries.

Streaming site plugins

Thirty-four ready-made **runtime source plugins** in `plugins/python/`. Each is a standalone process speaking the source-plugin protocol -- run it, register it, delete it whenever. Nothing is compiled into SPACE.

Movies & TV (TMDB-keyed)

Plugin	Port	Streams returned
`zstream_plugin.py`	8904	Z-Stream watch pages (`/media/tmdb-movie-...`)
`cineby_plugin.py`	8905	VidKing embed players + Cineby page
`toustream_plugin.py`	8906	TouStream players (`/tou/movies/...`, `/tou/series/...`)
`rivestream_plugin.py`	8903	RiveStream players ([docs](rivestream.md))
`cubeflix_plugin.py`	8920	Cubeflix (P-Stream clone) watch pages · **empty query -> TMD
`cinebytv_plugin.py`	8921	VidKing player + CinebyTV page · **empty query -> TMDB trend

Anime (AniList-keyed)

Plugin	Port	Streams returned
`miruro_plugin.py`	8907	**real HLS m3u8** per episode + watch page
`animepahe_plugin.py`	8908	play/all-episode pages
`kaa_plugin.py`	8909	site handoff links (see capability tiers)
`animex_plugin.py`	8910	site handoff links
`anikoto_plugin.py`	8911	site handoff links
`reanime_plugin.py`	8912	site handoff links
`lunastream_plugin.py`	8922	info-page streams · **empty query -> their trending feed**

Asian dramas & encodes

Plugin	Port	Search	Streams returned
`kissasian_plugin.py`	8914	? WP REST API	drama/episode pages
`dramanice_plugin.py`	8916	? WP REST API	drama pages
`pahe_plugin.py`	8919	? WP REST API	encode post pages
`kisskh_plugin.py`	8915	? JSON API	drama page (`/Drama/{slug}?id=`)
`goplay_plugin.py`	8913	--	passthrough by url
`asiaflix_plugin.py`	8917	--	passthrough by url
`asiancrush_plugin.py`	8918	--	passthrough by url
`flicker_plugin.py`	8923	--	passthrough by url (obfuscated client)

These frontends are unlicensed unless noted (AsianCrush is a legit ad-funded service) -- what you play is on you. Keep them on your homelab.

Shared behavior

- All speak `/manifest.json`, `/items` (health probe), `/search?q=`, `/details?id=` -- after registering they join unified search, playback resolution with failover, and health monitoring automatically.
- TMDB-keyed sites bridge search through **TMDB** using `TMDB_API_KEY` (env or the SPACE `.env`). Ids: `movie:{tmdb}` / `tv:{tmdb}[:s]:[e]`.
- Anime sites key on **AniList** (public GraphQL, no key): `anilist:{id}[:ep]`.
- Three drama sites are WordPress and searched through their own `/wp-json/wp/v2/search`; ids are canonical post urls (mirror hosts

such as `wwv19.kissasian.com.lv` are accepted automatically).

Source-only mode (recommended for these sites)

By default a registered source also appears in unified search (`/api/search/all`). For these streaming-site plugins you usually want the opposite: **clean search, sources asked only at play time.** Register with `resolve_only` and use the lookup endpoint:

```
# 1) register as resolve-only
curl -X POST http://localhost:8080/api/admin/plugins \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"name":"kissasian","kind":"source",
       "base_url":"http://127.0.0.1:8914","enabled":true,
       "config":{"resolve_only":true}}'

# 2) pick something via your normal catalog/metadata search, then:
curl -X POST http://localhost:8080/api/playback/lookup \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"title":"Vincenzo","year":2021,"category":"series"}'
# -> {"items":[{"token":"...","source":"kissasian","title":"Vincenzo ..."}]}

# 3) play exactly as always:
curl -X POST http://localhost:8080/api/playback/resolve \
  -H "Authorization: Bearer $TOKEN" -d '{"token":"<token from lookup>"}'
```

Resolve-only sources are excluded from `/api/search/all` fan-out entirely but are fully included in `POST /api/playback/lookup` (title-matched, ±1 year, category-filtered) and in resolution failover.

Anime / manga / torrents (direct APIs)

Plugin	Port	Data source	Streams
`nyaa_plugin.py`	8924	Nyaa **RSS feed** (official machine	**magnet** torrents
`mal_plugin.py`	8925	Jikan REST (MyAnimeList anime+manga	MAL info page
`weebcentral_plugin.py`	8928	server-rendered `/search/data` fragments	series pages
`simkl_plugin.py`	8927	official Simkl API (`SIMKL_CLIENT_ID`	simkl app page
`rottentomatoes_plugin.py`	8926	RT APIs are CF-walled -> passthrough	known urls

Passthrough tier (resolve known urls)

`comix`(8929) · `mangadot`(8930) · `noxx`(8931) · `bstsrs`(8932) · `primewire`(8933) · `hanime`(8934, 18+) · `comicbookplus`(8935) · `allcinema`(8936) · `everythingmoe`(8937)

Search dedupe (two providers, one result)

Unified search already collapses duplicates: results with the same normalized title + year + category merge into a single item whose `sources` array lists every contributing provider, keeping the best-ranked token. Title-based lookup matching (`POST /api/playback/lookup`) uses the same normalization, so "Vincenzo" from two providers never yields two rows. Metadata *imports* dedupe separately via external-id mappings.

Capability tiers (honest limits)

- **Full search + resolve**: zstream, cineby, toustream, rivestream, miruro, kissasian, dramanice, pahe, kisskh.
- **Search via AniList + site-handoff streams**: kaa, animex, anikoto, reanime -- JS-only frontends without stable deep-link formats; resolution opens the site's search pre-filled with the exact title (one click). Know a deep-link format? Set e.g. `KAA_WATCH_TEMPLATE="{site}/watch/{anilist_id}"` and real deep links are emitted instead.
- **Passthrough only**: goplay (origin unreachable at build time), asiaflix & asiancrush (app-locked APIs) -- resolve

known item urls into tokens; search reports empty until an upstream endpoint exists.

Run & register

```
python plugins/python/zstream_plugin.py     # repeat per plugin, ports above

curl -X POST http://localhost:8080/api/admin/sources/register-url \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"url": "http://127.0.0.1:8904"}'     # once per running plugin
```

Env knobs

Plugin	Vars
zstream / cineby / rivestream	`TMDB_API_KEY`, site-base override
toustream	`TOUSTREAM_BASE`, `TOUSTREAM_SERIES_TEMPLATE`
miruro	`MIRURO_API_BASE`, `MIRURO_CATEGORY` = `dub`
animepahe	`ANIMEPAHE_BASE`
kaa / animex / anikoto / reanime	`<NAME>_BASE`, `<NAME>_WATCH_TEMPLATE`
kissasian / dramanice / pahe / kisskh / goplay / asiaflix /	`<NAME>_BASE` mirrors

Notes

- Sites change often -- every URL template lives in one small file you can edit without touching the server.
- For direct files pair with the 1337x torrent plugin; pahe posts are download-oriented enco

des.

Extension system -- Stremio addons, 1:1

SPACE speaks the Stremio Addon SDK
protocol natively. Install any addon from the stremio ecosystem by its
manifest url and it becomes a first-class source: searchable catalogs join
unified search, `/stream` endpoints join playback resolution with failover,
and subtitles aggregate across all installed addons.

This is a **separate system** from SPACE's plugin/hub/web-source stacks --
different table (`extensions`), different namespace (`/api/extensions/*`,
`/api/admin/extensions`), different manager.

The protocol (implemented 1:1)

```
GET {base}/manifest.json
GET {base}/catalog/{type}/{catalogId}/{encoded extras}.json   e.g. search=batman&skip=100
GET {base}/stream/{type}/{id}.json        id: tt1254207 | tt123:1:2 | tmdb:... | mal:...
GET {base}/meta/{type}/{id}.json
GET {base}/subtitles/{type}/{id}.json?extra=...
```

Stream shapes mapped to SPACE kinds: `url` -> mp4/hls/page (sniffed),
`infoHash` -> **torrent magnet**, `ytId` -> YouTube page.

Install (admin console -> Extensions tab, or REST)

```
curl -X POST http://localhost:8080/api/admin/extensions \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"url":"https://torrentio.strem.fun/manifest.json"}'
```

The exact Stremio flow works: configure an addon on its website, copy the
resulting manifest url, paste it here. Well-known starting points:
Cinemeta `https://v3-cinemeta.strem.io/manifest.json`,
Torrentio `https://torrentio.strem.fun/manifest.json`.

Browse community addons

`GET /api/admin/extensions/discover?q=torrent` proxies the central Stremio
index so you can one-click-install from the console tab.

Management (localhost-only admin surface)

Endpoint	Purpose
`GET/POST /api/admin/extensions`	list / install
`PATCH /api/admin/extensions/{id}`	enable/disable
`DELETE /api/admin/extensions/{id}`	remove

`GET /api/admin/extensions/discover?q=`	central index browser

Resource surface (any authenticated user)

1:1 passthrough to any installed addon:

```
GET /api/extensions/{addonId}/catalog/{type}/{catalogId}?search=&genre=&skip=
GET /api/extensions/{addonId}/stream/{type}/{id}
GET /api/extensions/{addonId}/meta/{type}/{id}
GET /api/extensions/subtitles/{type}/{id}      # fans out across all addons
```

Unified integration

- every catalog with `extraSupported: ["search", ...]` becomes a provider in `/api/search/all`; results are deduped/merged with everything else;
- every stream-capable addon is consulted during token resolution and title-based lookup (including stream-only addons like Torrentio that have no catalogs);
- series ids pass through verbatim -- clients may resolve `tt123:2:5` episode-level tokens direct

ly.

Config: `EXTENSION_TIMEOUT_SECS` (default 15).

Web Sources -- turn any site into a source with one field

Web sources are **learned** catalogs. You paste a site url, SPACE asks the bundled galaxy scraper to figure out how the site's search works and what an item page looks like, stores the resulting *recipe*, and the site immediately joins unified search (`/api/search/all`), playback resolution with failover, health monitoring and admin sync -- exactly like compiled-in sources.

Quick start

```
cargo build --release            # space server
cd ../galaxy && cargo build --release   # galaxy binary (sibling checkout)
```

SPACE finds `galaxy.exe` automatically in `./`, `./target/release/`, `../galaxy/target/release/`; or set `GALAXY_BIN` explicitly.

1. Teach it a site (all fields except `site` optional)

```
curl -X POST http://localhost:8080/api/admin/websources \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{ "site": "https://movies.example.com", "category": "movie" }'
```

That's it. Under the hood galaxy:

1. sniffs the site's homepage for a GET search form (falls back to probing common patterns `/search?q=`, `/?s=`, ...);
2. infers repeating result-card structure (row selector + title/link/poster/ overview fields) or falls back to a URL-shape patte

rn;

3. fetches one sample item page (yours via `sample_item`, else the first hit) and learns detail selectors + stream probes (video tags, embeds iframes, direct media links);
4. validates everything against the live pages before saving.

Optional overrides: `name`, `label`, `search_url` (`"https://site/search?q={query}"` -- must contain `{query}`), `sample_item`, `category` (`movie|series|book|music|manga`).

2. Use it like any other source

- `GET /api/search/all?q=...` -- learned sites fan out with everything else; tokens work on `POST /api/playback/resolve` and `POST /api/resolve`.
- `GET /api/sources/{name}/browse` -- empty-query "latest" feed.
- `POST /api/admin/sources/{name}/sync` -- import items into your catalog.
- Health circuit-breaker: dead sites are skipped automatically.

3. Manage

Endpoint	Purpose
`GET /api/admin/websources`	list + liveness
`PATCH /api/admin/websources/{id}`	`{label?, category?, enabled?}`
`DELETE /api/admin/websources/{id}`	remove
`POST /api/admin/websources/{id}/relearn`	re-run learning (sites change!)
`POST /api/admin/websources/{id}/test`	live search + first-item resolve
`GET /api/admin/websources/{id}/recipe`	raw recipe JSON

The admin console has a **Web Sources** tab covering all of the above.

How it works at runtime

Recipes replay deterministically through the local galaxy binary (`galaxy source search|item --recipe -`, JSON over stdin/stdout). No AI is involved after learning; failures surface as `503 unavailable` and feed the source circuit breaker. Recipes are plain JSON -- inspect them with `galaxy source test` or fix them by hand in the DB if a site redesigns.

Source hubs -- connect servers together

Any SPACE instance can act as a **hub** for others: it serves `/api/hub/*` from its learned sources, accepts uploaded recipes, and answers find/item queries when a peer comes up empty. This is the "server asks that API" tier:

```
your laptop SPACE --search/resolve fallback--? hub SPACE --? learned sites
        ?                                                |
        +------------ push / pull recipes -----------+
```

- `GET  /api/hub/find?q=&category=&limit=` -- search across the hub's learned sources
- `GET  /api/hub/item?source=&id=` -- detail + streams (the "give it to me to watch" hop)
- `GET  /api/hub/sources` -- recipe catalog
- `POST /api/hub/sources` -- import a recipe into the hub

Auth: set `SOURCE_HUB_TOKEN` on the hub and send the same value as the `X-Hub-Token` header. With no token configured, `/api/hub/*` answers **loopback peers only**.

Client side (admin, localhost-managed like web sources)

Endpoint	Purpose
`POST /api/admin/hubs`	`{url, token?}` -- probes and registers
`GET /api/admin/hubs`	list + server-side status
`PATCH /api/admin/hubs/{id}`	`{url?, token?, enabled?}`
`DELETE /api/admin/hubs/{id}`	disconnect
`POST /api/admin/hubs/{id}/pull`	import the hub's recipes locally
`POST /api/admin/hubs/{id}/push`	upload your recipes to the hub

`POST /api/admin/hubs/{id}/test`	live find query

Once registered, a hub automatically joins unified search **and resolution failover**: when every local provider fails to produce a playable stream, the resolver asks your hubs before giving up -- results become watchable tokens like everything else. Dead hubs are skipped via the source circuit breaker.

Configuration

Env	Default	Meaning
`WEB_SOURCES_ENABLED`	`true`	master switch
`GALAXY_BIN`	auto-discover	path to the galaxy binary
`GALAXY_TIMEOUT_SECS`	`25`	per-invocation subprocess timeout (5-120)
`SOURCE_HUB_TOKEN`	unset	inbound auth for `/api/hub/*`; unset = loopback-only hub
`HUB_TIMEOUT_SECS`	`15`	outbound timeout when querying registered hubs

CLI equivalents (galaxy side)

```
galaxy source learn  --site https://site.example [--search-url ..] [--sample-item ..] [--category movie]
galaxy source search --recipe recipe.json "query" --limit 20
galaxy source item   --recipe - https://site.example/movie/x   # recipe on stdin
galaxy source test   --recipe recipe.json
```

Legal & etiquette

Learned sources scrape third-party sites: respect their terms and local law, and prefer official APIs/plugins when they exist.

Adding a Source

SPACE treats every catalog identically: search fans out to all healthy sources, results merge, and the central resolver picks streams at play time. There are **two ways** to plug into that, depending on where your code runs.

Way 1 -- Compiled-in source (Rust, `NativeSource`)

Best for: sources you maintain and want zero-hop in-process speed. Reference implementation: `src/sources/librivox.rs` (added as the worked example alongside this guide).

The whole contract is four required + three optional methods:

```
#[async_trait]
impl NativeSource for MySource {
    fn name(&self) -> &'static str { "mysource" }        // registry key
    fn label(&self) -> &'static str { "My Source" }      // display
    fn category(&self) -> &'static str { "book" }        // movie|series|book|manga|music|podcast|audiobook

    async fn search(&self, query: &str, limit: i32)
        -> ApiResult<Vec<SourceItem>>;                   // cards for /api/search/all
    async fn item(&self, external_id: &str)
        -> ApiResult<SourceDetail>;                      // detail + playable StreamLinks

    // optional overrides:
    fn capabilities(&self) -> SourceCapabilities { .. }  // tells the resolver what you support
    fn manifest(&self) -> serde_json::Value { .. }       // self-description for /api/sources
    async fn browse(&self, page: i32) -> Option<Vec<SourceItem>> { .. }
}
```

`StreamLink` is `{ label, url, kind }` where `kind` is one of `mp4 | hls | audio | page | file`. URLs must be directly fetchable -- that is what the resolver hands clients.

Registration (one line)

```
// src/sources/mod.rs -> SourceRegistry::new()
providers: vec![
    Box::new(archive::ArchiveFilms::default()),
    Box::new(librivox::LibriVox::default()),   // <- yours
    ...
]
```

That single registration wires you into everything automatically:

- `/api/search/all` fan-out with per-source timeouts and duplicate merging
- circuit breaker (3 failures -> skipped) + 60 s half-open prober

- telemetry on `/api/sources/status` (latency, success rates, active requests)
- resolver preference ranking + title-based failover
- admin sync-to-catalog imports and debug endpoints

Way 2 -- HTTP plugin (any language)

Best for: third-party or dynamically deployed catalogs. SPACE speaks a tiny JSON protocol:

Route	Response
`GET /manifest.json`	`{ name, label, category, transport }`
`GET /items?limit=`	`{ items: [SourceItem] }`
`GET /search?q=&limit=`	`{ results: [SourceItem] }`
`GET /details?id=`	`{ title, overview?, year?, poster?, media_type, streams: [{`

Register without touching the server binary:

```
POST /api/admin/sources/register-url
{ "url": "https://my-plugin.example.com" }
```

SPACE probes the three routes and registers whichever protocol answers.

Python? A runnable stdlib-only template lives at `plugins/python/space_plugin.py` -- see `docs/plugins-python.md`.

Rules that keep a source in good standing

1. **Legality**: catalogs must be public-domain, openly licensed, or authorized. Sources serving unlicensed copies of copyrighted works are not accepted -- this is a hard project rule, not a style preference.
2. **Direct URLs**: every `StreamLink.url` must be playable without extra scraping

hops at request time.

3. **Timeouts matter**: fan-out allows 9 s per provider; slow sources get dropped from results and their success rate suffers.
4. **Failures are cheap**: return `Err` early rather than empty success -- the breaker uses outcomes to protect the rest of the platform.

Telemetry & Monitoring

architecture.md · deployment.md · API.md

SPACE exports Prometheus metrics and structured logs for production observability.

Enabling Metrics

Set `METRICS_ENABLED=true` in your environment (default: `true`).

The `/metrics` endpoint serves a standard Prometheus text format. Scrape it with your Prometheus instance, Grafana Agent, or any OpenMetrics-compatible collector.

Structured Logging

Variable	Default	Effect
`LOG_LEVEL`	`info`	tracing filter: `debug`, `info`, `warn`, `error`
`LOG_JSON`	`false`	`true` -> JSON log lines (machine-parseable)
`RUST_LOG`	--	Override for any crate

JSON mode outputs one JSON object per line with fields: `timestamp`, `level`, `target`, `message`, and any structured fields attached via `tracing::info!(field = value, ...)`.

Prometheus Metrics

All metrics use the `metrics` crate with `metrics-exporter-prometheus`.

Application Metrics

Metric	Type	Labels	Description
`http_requests_total`	counter	`method`, `path`, `status`	Total HTTP requests (auto-instrumented by `axum-metrics`)
`http_request_duration_seconds`	histogram	`method`, `path`	Request latency distribution
`playback_blocked_no_card_total`	counter	--	Playbacks rejected for missing access card
`cache_warmed_total`	counter	--	Cache entries warmed per warming tick
`cache_warm_duration_seconds`	histogram	--	Wall-clock time of warming tick
`automation_runs_total`	counter	`event`	Automation rules executed
`webhook_deliveries_total`	counter	`event`, `status`	Webhook delivery attempts
`cards_minted_total`	counter	`issued_by`	Cards minted (`admin`, `trial`, `merchant`)
`card_activations_total`	counter	`result`	Activation attempts (`success`, `failure`)

Infra Metrics (auto-exported by runtime crates)

Metric	Source	Description
`sqlx_pool_*`	sqlx	Connection pool: size, idle, wait, timeout
`tokio_*`	tokio	Async runtime: threads, tasks, queue depth

Cache Warming

A background task runs every 10 minutes (`src/warming.rs`):

1. **User rails**: For up to 50 online users, precomputes recommendation caches (12 items each)
2. **Trending shelves**: For the top 20 trending media of the week, precomputes similar-media caches (8 items each)

Warming is invisible to users -- they just see fast rails. The warming loop respects the `CancellationToken` for graceful shutdown.

Metrics emitted:

- `cache_warmed_total` -- number of cache entries written
- `cache_warm_duration_seconds` -- wall-clock time of the tick

Health Endpoints

Endpoint	Purpose	Returns
`GET /health`	Liveness probe	`200 "ok"`
`GET /api/ready`	Deep readiness	`200 { "status": "ok", "checks": { "database": true, "redis"`
`GET /api/stats`	Dashboard summary	Session counts, transcode queue, media counts
`GET /metrics`	Prometheus scrape	Text format, all registered counters/histograms
`GET /api/admin/events`	Event timeline	Platform events with timestamps
`GET /api/admin/failures`	Error tracker	Recent 5xx errors, per-route aggregates

Alerting Suggestions

Critical

- `playback_blocked_no_card_total` spike -> card system outage
- `http_requests_total{status="503"}` -> site lockdown active or upstream down
- `sqlx_pool_*` wait timeout spike -> database overloaded

Warning

- `webhook_deliveries_total{status="network_error"}` -> webhook endpoint unreachable
- `cache_warm_duration_seconds` > 30s -> recommendation engine slow
- `http_request_duration_seconds` p99 > 5s -> general latency degradation

Info

- `automation_runs_total` -> automation activity level
- `cards_minted_total` -> merchant/admin card activity

Grafana Dashboard

A minimal dashboard requires:

1. Prometheus data source pointed at `GET /metrics`
2. Panels for: request rate (`rate(http_requests_total[5m])`), error rate, latency histogram, card blocks, cache warming duration

Graceful Shutdown

On `SIGTERM`:

1. Stop accepting new connections
2. Drain in-flight HTTP requests (up to 30s)
3. Close all WebSocket connections with `Close` frames
4. Cancel the cache warming loop
5. Close the database pool

The `/health` endpoint returns `503` once shutdown begins, allowing load balancers to stop routing traffic.

Torrents -- the 1337x source plugin

1337x support ships as a **runtime source plugin** -- a standalone process speaking the source-plugin protocol. Nothing is added to the server binary: run the script, register it, delete it whenever you like.

> *__Legal.__ Torrents are P2P metadata links. 1337x indexes both legal and infringing content -- what you fetch is on you. Keep this on your homelab, respect your local law, and seed responsibly.*

Run it

```
python plugins/python/leetx_plugin.py --port 8902
# -> serving 1337x · Torrents on http://127.0.0.1:8902
```

Stdlib only -- no pip installs. Endpoints: `/manifest.json`, `/items` (health probe), `/search?q=`, `/details?id=` (extracts the **magnet link**).

Register with SPACE

Either paste-and-probe:

```
curl -X POST http://localhost:8080/api/admin/sources/register-url \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"url": "http://127.0.0.1:8902"}'
```

or explicitly:

```
curl -X POST http://localhost:8080/api/admin/plugins \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"name":"leetx","kind":"source","base_url":"http://127.0.0.1:8902","enabled":true}'
```

Once registered it joins everything automatically:

- unified search fans out to it (`GET /api/search/all`),
- play tokens resolve through `/details`, returning `kind:"torrent"` streams (ranked as last resort below any direct mp4/HLS from other

sources),

- health probing keeps it skipped when the site or plugin is down,
- `POST /api/admin/sources/leetx/sync` works too (imports nothing -- torrents aren't library items).

Getting blocked? Transport chain

The plugin tries, in order:

1. **direct** HTTPS fetch,
2. **FlareSolverr** when `FLARESOLVR_URL=http://localhost:8191` is set,
3. your local **galaxy** binary (auto-discovered next to the repo, or `GALAXY_BIN=/path/to/galaxy.exe`) -- set `GALAXY_STRATEGY=auto` to allow its stealth headless-browser escalation for hard Cloudflare days.

Other env knobs: `LEETX_BASE=https://mirror.example` for proxies/mirrors.

Watching results

Resolution hands clients a `magnet:` url. Feed it to whatever you run locally, e.g. qBittorrent's WebUI (`Add torrent -> URL`), or wire download automation: SPACE's automation rules can react to `media.added` events, and the resolved stream is available via `POST /api/playback/resolve {token}` for scripts.

RiveStream source plugin

RiveStream support ships as a **runtime source plugin** -- a standalone process speaking the [source-plugin protocol](plugins.md). Nothing is compiled into the server binary.

RiveStream publishes a public embed API keyed by **TMDB ids** (movies, TV seasons/episodes, torrent-backed playback, aggregator servers, downloads).
This plugin bridges SPACE onto it:

- `/search?q=` -> proxied through **TMDB multi-search** so normal queries work (`TMDB_API_KEY` from your `.env` is reused automatically)
- `/details?id=` -> returns four `kind:"page"` streams per title:

Stream	Endpoint
Rive · fastest server	`/embed?type=...&id=...`
Rive · aggregator servers	`/embed/agg?...`
Rive · torrent stream	`/embed/torrent?...`
Rive · download	`/download?...`

> *Unlicensed streaming service: what you play is on you. Keep it homelab.*

Run it

```
python plugins/python/rivestream_plugin.py --port 8903
```

Stdlib only. Env knobs:

Var	Meaning
`TMDB_API_KEY`	enables search; auto-read from the SPACE `.env` when unset
`RIVESTREAM_BASE`	site base override, default `https://www.rivestream.app`

Register

```
curl -X POST http://localhost:8080/api/admin/sources/register-url \
  -H "Authorization: Bearer $TOKEN" \
```

```
  -d '{"url": "http://127.0.0.1:8903"}'
```

or explicitly `POST /api/admin/plugins {"name":"rivestream","kind":"source", "base_url":"http://127.0.0.1:8903","enabled":true}`.

Afterwards: unified search includes it, tokens resolve to its players, and health probing skips it when it's down.

Id format (for scripts / manual calls)

```
movie:{tmdb_id}            e.g. movie:278        (The Dark Knight)
tv:{tmdb_id}               defaults to S1E1      e.g. tv:1396       (Breaking Bad)
tv:{tmdb_id}:{s}:{e}       explicit episode      e.g. tv:1396:2:7
```

Without a TMDB key, `/details` still resolves ids to players -- only search needs the key.

PLAN -- Unified Media Resolution Pipeline + Security Layer + Admin AI Layer

Status: **A-J + hardening shipped** · **changeset 5 shipped** · **changeset 6 shipped** -- completed/toplists/random/upcoming/availability/summary/export/meta QoL APIs, plus the full security audit (`SECURITY-AUDIT.md`).

Shipped in changeset 5

Area	Delivered
Moderation	`POST /api/reports` mute`, delete+lists) enforced in social feeds and DMs
Chat	message edit (owner, `PATCH`) and soft delete (`DELETE`, own
Presence	`PUT /api/me/presence` (online/idle/watch_party/dnd/invisibl
Audit	`audit()` helper at every sensitive action; `GET /api/admin/
DX	WS schema doc finalized; unified-search result cache (~90 s
Admin fortress	TOTP 2FA for admins (RFC-6238, provisioning URI); behavioral
Card economy	Cryptographic access cards (HMAC checksum numbers). No activ

Shipped in changeset 4 (anti-theft hardening)

Feature	Delivered
Device/TLS fingerprinting	Grants bind to a client-context hash: edge-computed `X-JA4`
Segment-scoped grants	Parent grant mints child tickets scoped to `seg:<index>` (TT
Offline keys	`POST /api/playback/offline-keys` issues a 24 h license: AES

Shipped in changeset 3 (phase J + ops hardening)

Phase	Delivered
J	Key service (`src/security/keys.rs`): master secret (env `MA
Origin	`ORIGIN_INTERNAL_TOKEN`: when set, key/download endpoints re
Failure analysis	5xx recorder middleware -> `failure_events`, grouped timelin
Adaptive privacy	maintenance tick rolls up platform events older than 30 d in
Self-organizing library	`POST /api/admin/quality/duplicates/auto-resolve`: merges du

Shipped in changeset 2 (phases F-I)

Phase	Delivered

F	`media_relationships` graph (12 relation types incl. adaptat
G	Universal progress engine: `unified_progress` view (seconds
H	Event bus (`events` table + WS bridge envelopes on the globa
I	Proper WS: server heartbeat (30 s ping, dead-connection reap

Shipped in changeset 1 (phases A-E)

Phase	Delivered
A	`NativeSource.capabilities()/manifest()/browse()` defaults;
B	`POST /api/resolve`: identify -> circuit-aware providers ->
C	Playback Security Controller: HMAC-SHA256 grants (instant re
D	AI Copilot: heuristic + OpenAI-compatible providers, persist
E	Migration 0014, env vars, docs/API.md §15-17, admin UI (Secu

Changeset 3 design notes

```
KEY SERVICE                     WATERMARKING                  ORIGIN
MASTER_KEY env | generated      wm = hash(user,session,time)  X-Origin-Token header
  + HKDF purpose KEKs           appended ?wm=<code> on every    required on key/dl endpoints
      + AES-GCM wrapped media       resolved stream URL          (private origin behind CDN)
          keys (rotate/revoke)    leak -> traceable session
HLS: EXT-X-KEY -> /api/playback/keys/{file} verifies grant before releasing bytes

FAILURE ANALYSIS                ADAPTIVE PRIVACY              AUTO-MERGE
5xx -> failure_events            events >30d -> event_rollups   duplicates -> keeper =
timeline + p95 grouping         raw >90d deleted               most external mappings
AI rule: spike -> notify         security details scrubbed      (files/tracking moved,
                                                                conflicting series skipped)
```

Shipped in changeset 2 (phases F-I)

Phase	Delivered
F	`media_relationships` graph (12 relation types incl. adaptat
G	Universal progress engine: `unified_progress` view (seconds
H	Event bus (`events` table + WS bridge envelopes on the globa
I	Proper WS: server heartbeat (30 s ping, dead-connection reap

Shipped in changeset 1 (phases A-E)

Phase	Delivered
A	`NativeSource.capabilities()/manifest()/browse()` defaults;
B	`POST /api/resolve`: identify -> circuit-aware providers ->
C	Playback Security Controller: HMAC-SHA256 grants (instant re
D	AI Copilot: heuristic + OpenAI-compatible providers, persist
E	Migration 0014, env vars, docs/API.md §15-17, admin UI (Secu

Changeset 2 design notes

```
MEDIA GRAPH                     EVENT BUS                     UNIVERSAL PROGRESS
media_relationships             events table + broadcast      unified_progress VIEW
  adaptation_of ... parent_of     emit points:                  watch_history  -> seconds
BFS journey(depth<=3)               resolution.issued           reading_progress -> page/%
  related (grouped)                 playback.authorized       PUT /api/progress (routes by type)
  journeys (cross-media,            progress.updated          GET /api/continue (all types)
```

```
    adaptation chains)            security.*, search.performed

PERSONAL TWIN                     CACHE WARMER                    WEBSOCKET
twin computed from signals        every 10 min:                   heartbeat ping/pong + reaping
  genres · formats · completion     online users -> rec caches     user:{uuid} personal rooms
  pace · creators · discovery       trending -> similar caches     platform.event envelopes
daily snapshot persisted          metrics warmed_total            docs/websocket.md v2
```

Migration `0015_universal_progress.sql`: `media_relationships`, `events`, `twin_snapshots`, view `unified_progress`.

Later phase (deliberately out of scope)

- J: Forensic watermarking hook on stream serving + AEAD (AES-GCM) segment encryption behind the key service

---

Historical detail (changesets 1-2 planning)

```
Client --? GET /api/search/all            --? aggregator fan-out --? normalized items {id,title,type,sources[]}
Client --? POST /api/playback/session     --? Playback Security Controller
                                                +- authenticate
                                                +- risk assessment (optional layer, admin-tuned)
                                                +- decision: allow | challenge | revoke
                                                +- short-lived signed playback grant
Client --? POST /api/resolve              --? central resolver pipeline
        identify ? sources ? health ? candidates ? rank(prefs: quality/language/subs)
        ? select ? failover list ? grant-bound authorization ? playable stream (+expires_at)
Admin  --? /api/admin/security/*          --? policy, live events, instant revocation
Admin  --? /api/admin/ai/*                --? AI copilot: config, analyze, apply, audit
                                          (works with the security layer or fully without it)
Ops    --? GET /api/sources/status        --? per-source latency, search/resolve success rates,
                                             active requests, circuit state
```

Phase A -- Source abstraction + normalized results + telemetry

1. **Extend `NativeSource`** with defaulted async methods (no breakage for the 6 built-ins):
   - `capabilities()` -> `{categories, kinds, subtitles: bool}`
   - `manifest()` -> self-describing JSON (name/label/category/capabilities)
   - `browse(page)` -> optional catalog browse hook (default `None`)
2. **Normalize `/api/search/all` output** to the target contract:

```
{"it
```

```
ems":[{"id":"<opaque-token>","title":"...","type":"movie","year":2024,

"poster":"...","overview":"...","sources":["source_a","source_b"],

"relevance":98.5}],

"results":[...legacy array pres
```

```
erve
d
verb
atim
for
web/
app.
js
comp
at..
. ]}
```

- aggregator tracks the producing source per result and **merges duplicates** (same normalized title+year+category) into one item with `sources: []`.

3. **Telemetry upgrade** (`HealthRegistry`):

- per-op counters: `search_total/success`, `resolve_total/success`
- `active_requests: AtomicUsize` guard incremented around every provider call
- derived `success_rate`, `search_success_rate`, `resolve_success_rate`, `circuit` (`closed|open|half-open`)
- aggregator records resolve outcomes (currently never recorded)
- `/api/sources/status` returns the enriched snapshot

Phase B -- First-class central resolver: `POST /api/resolve`

Request:

```
{"token":"...","episode_id":null,
 "preferences":{"quality":"1080p","language":"en","subtitles":["en"],"direct_play":true}}
```

Pipeline implemented in `aggregator::resolve_with_preferences`:

1. parse token -> primary provider first, healthy providers after (circuit-aware)
2. fetch candidate streams; score each: quality number parsed from label/kind,

language match, subtitle availability hint, kind weight (hls > original > mp4...),

source health bonus (latency + success rate)

3. best candidate selected; remaining ranked streams become structured `fallbacks`
4. issue **short-lived playback grant** (Phase C) bound to user+media+quality+expiry;

response carries `authorization` + `expires_at`

5. response:

```
{"resolution_id":"...","title":"...","url":"...","kind":"hls","quality":"108
```

```
0p",

"aud
io":
"en"
,"su
btit
les"
:["e
n"],
"exp
ires
_at"
:"..
.","
auth
oriz
atio
n":"
<sig
ned>
",

"fal
lbac
ks":
[{"s
ourc
e":"
..."
,"ki
nd":
"mp4
","u
rl":
"...
"}]}
```

`/api/playback/resolve` stays as a thin backwards-compatible alias.

Preference scoring is a pure function -> unit-tested.

Phase C -- Playback Security Controller (risk layer)

New `src/security/` module:

- **Grants** (`grants.rs`): HMAC-SHA256-signed tokens reusing the `signing_secrets`

table (purpose `playback`), payload `gid|u

id|media|quality|exp`, constant-time verify, DB-backed **instant revocation** (`revoked_at`). TTL default 600 s (env `PLAYBACK_GRANT_TTL_SECS`, clamped 60-3600).

- **Risk engine** (`risk.rs`), off-by-default single-row policy table:
- signals: concurrent active sessions, distinct devices (24 h), resolutions/hour, repeated challenge history
- weighted score -> decision `allow | challenge | revoke`
- thresholds admin-tunable; every non-allow decision written to `security_events`
- **Endpoints**:
- `POST /api/playback/session` -- controller entry: auth -> risk gate -> grant (`403 challenge_required` on suspicious; clients retry with `acknowledge_challenge:true` which is logged)
- `GET/PATCH /api/admin/security/policy`
- `GET /api/admin/security/events`
- `GET /api/admin/security/grants` + `POST /api/admin/security/grants/{id}/revoke`
- **Layer toggle**: `risk_policies.enabled = false` ? `/api/resolve` and the controller pass straight through (the AI layer

works identically either way).

- Migration `0014_ai_security.sql`: `playback_grants`, `security_events`, `risk_policies` (+ Phase D tables).

Phase D -- Admin AI layer ("AI Copilot")

New `src/aiops/` module -- pluggable so the real brain can be added later:

- **Providers** (`provider.rs`): trait `AiProvider::analyze(ctx) -> Vec`
- `heuristic` (built-in, zero-config): rule-based over live context (dead/degraded sources, low success rate, stuck transcodes, risk spikes, empty library, backlog...) -> concrete recommendations
- `llm` (optional): OpenAI-compatible `/chat/completions` (`AI_API_URL` / `AI_API_KEY` / `AI_MOD

EL` env defaults, admin-editable);

failures degrade gracefully to heuristic

- **Governance** (`engine.rs`): persisted config `ai_config`
- `mode`: `off | heuristic | llm`
- `autonomy`: `observe` (log only) · `suggest` (admin clicks Apply) · `act` (background loop auto-applies whitelisted actions)
- `scopes`: `["sources","security","library","notifications","grants"]`
- `security_integration`: bool -- when false the AI never sees/touches the security domain (works without the layer)
- **whitelisted actions only**: toggle source plugin, patch risk policy fields, revoke grants (per user/all), notify admins, run library scan --

every apply gated by scope + autonomy + written to `ai_events`

- **Endpoints** (all admin-gated):
- `GET|PATCH /api/admin/ai/config`
- `POST /api/admin/ai/analyze` -> context summary + recommendations
- `POST /api/admin/ai/apply` `{recommendation_id}` (manual, any mode != off)
- `GET /api/admin/ai/events`, `GET /api/admin/ai/status`
- Background actor loop spawned from `AppState` (cancel-token aware) when `mode != off && autonomy == act`.
- **Admin UI**: new "AI Copilot" tab (config form + Analyze + recommendation cards with Apply + audit log) and new "Security" tab (policy editor, live events, grant list with revoke buttons).

Phase E -- Wiring + docs + verification

- `state.rs`: build `AiEngine` handle, spawn actor loop
- `router.rs` / `main.rs` / `docs.rs` (OpenAPI): register everything
- `.env.example`: AI + playback-grant vars
- `docs/API.md`: new endpoints section
- Verify: `cargo check`, `cargo test` (unit tests: preference ranking, duplicate merge, risk scoring band

s, grant sign/ verify /expir y/rev oke), then `carg o build`

Later phases (deliberately out of this changeset, follow the plan file)

- F: Universal media graph (`media_relationships`, canonical identity, aliases)
- G: Universal progress engine across video/audio/text (single `MediaProgress`)
- H: Forensic watermarking hook on stream serving + AEAD (AES-GCM) segment encry ption behin d the key servi ce
- I: Event bus extraction (`events` table + subscribers) replacing direct webh ook/a utom ation dispa tch calls
- J: Predictive cache warming / demand prediction feeding off the event bus